

UNIVERSITY OF
Waterloo



Department of Mechanical and Mechatronics Engineering

ME101: Lego EV3 Pancake Printer Report

A Report Prepared For:

ME101 – Professor Peter Teertstra

Prepared By:

Calim Kim-Wansbrough

Dylan Lam

Nathan Ma

Marwan Ziyadeh

21143924

21162063

21122424

21155732

April 4th, 2025

Table of Contents

Table of Contents	i
List of Figures	v
List of Tables	vii
Summary	viii
1. Needs Analysis.....	1
1.1. Background	1
1.2. Needs Statement	2
1.3 Engineering Specifications and Criteria.....	2
2. Conceptual Design	3
2.1. Conceptual Morphological Designs	3
2.2. Decision Making Matrix	7
2.3. Final Conceptual Design	9
3. Mechanical Design.....	9
3.1. Layout.....	9
3.1.1. Frame Design.....	10
3.1.2. Extrusion and Drive Train	14
3.1.3. EV3 Brick and Sensors.....	20
3.1.4. Fabrication	22

4. Verification of Design.....	24
4.1. Verification of Specification	24
4.1.1. Difficulty of Operation	24
4.1.2. Consecutive Uses.....	25
4.1.3. Dimensions	25
4.1.4. Mass.....	25
4.1.5. Dispensable Batter	26
4.1.6. Printable Area	26
4.1.7. Deformation of Fabricated Parts.....	26
4.2 Overall Assessment	27
5.0. Software Design and Implementation.....	27
5.1. Description of Software	27
5.1.1. Flow Charts.....	28
5.1.2. Task List	29
5.1.3. Functions	32
5.1.3.a float getY() – Marwan Z.	32
5.1.3.b float getX() – Marwan Z.	32
5.1.3.c void extrude(bool on) – Nathan M.	32
5.1.3.d void move_Both(float x, float y, float ex_minX, float ex_maxX, float ex_minY, float ex_maxY) – Calim KW.	32

5.1.3.e void prepare(int x, int y) – Calim KW.....	33
5.1.3.f void reset() – Calim KW.	33
5.1.3.g void square() – Calim KW.....	33
5.1.3.h void triangle() – Calim KW.....	33
5.1.3.i void spiral() – Dylan L.	34
5.1.3.j void manual(bool & on) – Nathan M.	34
5.1.3.k void colorShape(bool & system) – Dylan L.	34
5.1.3.l int confirm(string message) – Nathan M.	35
5.1.3.m task main() – All.....	35
5.1.3.n Function Design.....	35
5.1.4. Data and Trade-Offs	39
5.2. Testing Program	39
5.3. Programming Problems.....	41
6. Project Management	42
6.1. Work Break Down	42
6.2. Project Risk Assessment	44
6.3. Progress Schedule	47
6.4. Actual Timeline.....	49
7. Conclusion	51
7.1 Mechanical Portion	51

7.2 Programming Portion	52
8. Recommendations	53
8.1. Mechanical Design Recommendations	53
8.2. Software Design Recommendations	54
Appendix A – Pancake Printer Code	56

List of Figures

Figure 1	Conceptual design #1	5
Figure 2	Conceptual design #2	6
Figure 3	Conceptual design #3	7
Figure 4	SolidWorks Visualization of Final Design (Front and back view)	10
Figure 5	SolidWorks Assembly of Final Designed Frame.....	10
Figure 6	Dimetric View SolidWorks Assembly of Fully Designed Nozzle Extrusion Cart Sitting on X-Direction Rail Gantry and Y-Direction Rail.....	11
Figure 7	Pin Used to Mount X-Axis Rail to Y-Axis Rail.....	12
Figure 8	SolidWorks Assembly of Central Rail Mounting Brackets.....	13
Figure 9	Dimetric View SolidWorks Assembly of Fully Designed Nozzle Extrusion Cart Sitting on X-Direction Rail Gantry and Y-Direction Rail.....	14
Figure 10	Batter Extrusion Mechanism.....	15
Figure 11	Cross Sectional View of Extrusion Motor Bracket.....	16
Figure 12	Bracket Mounting Syringe to Beam.....	17
Figure 13	Frame With Y-Axis Wheels Mounted.....	18
Figure 14	Y-Axis Motor Drive System in Assembly.....	19
Figure 15	Top View of Assembled Robot Showing Cart.....	19
Figure 16	Front Facing View of Final Design with EV3 Brick, Touch Sensor, and Colour Sensor.....	20
Figure 17	Brackets Mounting the Brick and Sensors Onto the Frame.....	20
Figure 18	Laser Cut Rails shown in Full Rail Assembly.....	22

Figure 19	Printing Area Including Printing Tray Supports and Connected Syringe.....	23
Figure 20	Diagram of Full Programming Flow Chart.....	24
Figure 21	Code of Function Responsible for Moving Nozzle Cart.....	28
Figure 22	Code of Colour-Based Shape Printing.....	36
Figure 23	Image of Main Function with User Instruction.....	38
Figure 24	Image of Confirm Function with User Message.....	38
Figure 25	Project Schedule of Original Timeline.....	47
Figure 26	Project Timeline a Week Prior to Live Demonstration.....	48
Figure 27	Project Timeline Day of Live Demonstration.....	49

List of Tables

Table 1	Engineering Design Specification	2
Table 2	Morphological Chart	4
Table 3	Decision-Making Matrix	7
Table 4	Work Break Down Structure of Project Along with Respective Assignee.....	42
Table 5	Project Assessment of Technical and Non-Technical Risks.....	45

Summary

The team developed a pancake printing robot for their course project. The robot was designed to create pancake designs of the user's choosing. Design options for the user included a spiral, triangle, and square. In addition, the robot offers the user to print pancakes in a manual mode where the user can control where the robot extrudes batter using buttons.

Initially, the robot was designed to print a pancake in the shape of a face, however, the design ran into some issues regarding the viscosity of the pancake. The pancake batter would often keep extruding after the extrusion had stopped. As a result, drawing non-continuous shapes were difficult to manufacture and as a result, the face design was replaced with a spiral. The issue with the viscosity of the batter also required redesigns. For one, a new extrusion system involving a lead screw had to be designed to replace the weaker rack and pinon system to extrude batter.

Various sensors including the motor encoders, touch sensor, brick, and colour sensor was implemented to operate the robot. The motor encoders were used as position trackers and the colour sensor, touch sensor and integrated brick buttons were used to interface with the user. The colour sensor read a colour displayed by the user and drew a specific shape based on the colour. The robot involved the use of the console on the brick to effectively communicate with the user, ensuring that they wouldn't get lost at any point of operating the robot.

To ensure that the robot met all the criteria, the team conducted numerous tests. Iteration through testing allowed the group to improve upon the design and meet the detailed requirements of the robot.

1. Needs Analysis

1.1. Background

In recent years, the intersection of food and robotics has become an increasingly popular area of innovation. Researchers and engineers are always exploring innovative technologies and techniques to automate kitchen appliances, prepare, and present food. One such innovation is the pancake printer, a device that combines robotics, software, and pancakes that are printed with precision and consistency.

Pancakes are traditionally made by pouring batter onto a hot griddle or pan creating circular shapes. One inherent problem with pancakes is that they all have the same circular shape. A pancake printer leverages principles of additive manufacturing, commonly found in 3D printing, to dispense pancake batter into intricate custom designs. Not only does this enhance the visual presentation of pancakes but adds an individualized touch to the presentation making it perfect for home kitchens, catering services, and restaurants alike.

Furthermore, the applications of pancake printers extend far beyond boosting the presentation of the dish. They can also serve as a powerful educational tool to engage children in STEM topics like programming, robotic control, and mechanical design, through hands-on learning. Through the highly abstracted code and use of EV3 Mindstorms parts, kids can explore concepts like coding, robotics, and spatial reasoning. This process bridges abstract STEM ideas with edible outcomes, fostering creativity and problem-solving skills while making learning fun and memorable. The pancake printer thus represents not just a novel gadget, but a tool for inspiring future innovators at the intersection of food, art, and technology. This project aims to

create a prototype pancake printer based on LEGO EV3 electronics, Tetrix pieces, and miscellaneous rapid prototyped pieces.

1.2. Needs Statement

A need exists for a robot that automatically extrudes pancake batter on a two-dimensional plane while weighing less than 6 kilograms.

1.3 Engineering Specifications and Criteria

No.	Characteristic	Relation	Value	Units	Verification Method	Comments
1	Difficulty of Operation	N/A	N/A	N/A	Demonstration	Ask a student to try the robot. Passes if they can successfully print the design that they wanted to whether it be using a preprogrammed design or manual mode.
2	Consecutive Uses	$\geq$	3	Uses	Test	Run it 3 consecutive times. Inspect for malfunctions.
3	Width, Length, Height	$<$	50, 50, 40	cm	Test	Measure dimensions using a tape measurer.
4	Mass	$\leq$	6	kg	Test	Weigh the final product on a scale.
5	Volume Dispensable	$\geq$	75	mL	Test	Measure the volume difference between a full syringe and the remaining amount when it can no longer extrude.
6	Printable Area	$\geq$	225	cm ²	Test	Measure the maximum displacement of the nozzle in both x and y

						directions. Multiply to calculate printable area.
7	Components resist deformation	N/A	N/A	N/A	Examination	Visually examine 3D printed parts for potential warping due to stress/heat.

Table 1: Engineering Design Specification

These engineering specifications have been derived from the most essential functional, non-functional, and constraint requirements. The values related to each characteristic are developed based on the verification of the minimum viable product. The core specifications ensure that the printer is easy to operate, reliable, lightweight, small, structurally sound, and capable of dispensing a certain amount of batter.

2. Conceptual Design

2.1. Conceptual Morphological Designs

The conceptual design of the robot is integral to the overall success of the design. For a complex multifaceted project like a pancake printer, one must break the robot down into its most essential subfunctions. When broken up this way, each subfunction can be considered individually. This is the approach that is taken in the morphological chart found in Table 2. The three main subfunctions to be considered individually were a mechanism responsible for a nozzle movement, a user interface, and a batter extrusion system.

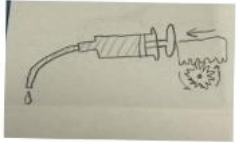
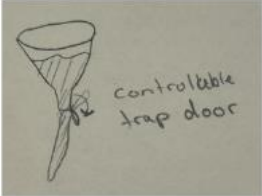
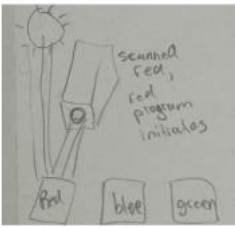
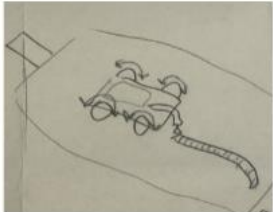
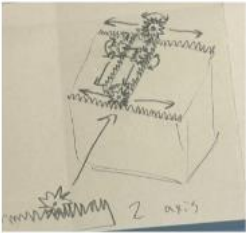
	1	2	3	4
Batter Extrusion Mechanism	Syringe that is pushed by a rack and pinion 	Piping Bag, rod spins squeezing the batter 	Funnel released with a trapdoor, gravity 	
Input	User presses buttons determining the input. 	Colour Detection. Different colours are read as different inputs. 	Touch sensors are set to corresponding inputs. 	Motor's encoder used as a turn knob to select the print 
Nozzle movement	Whole product sits on a car that drives around 	Motors running along two linear axes. To span a plane. Each axis runs on rack and pinions 	2 axis rails, wheels on rails. Motors controlling wheels 	

Table 2: Morphological Chart

This morphological chart was used to create two gantries-based designs and a car design. The first conceptual design was centred on the use of rack and pinion-based gantries, pictured in Figure 1. The extrusion mechanism consists of a piping bag filled with batter, which is compressed by a motor-driven rolling system to dispense the batter. The printer accepts user

input in the form of a coloured piece of paper presented to the colour sensor. It then reads the colour sensor input and prints a design corresponding to the detected color.

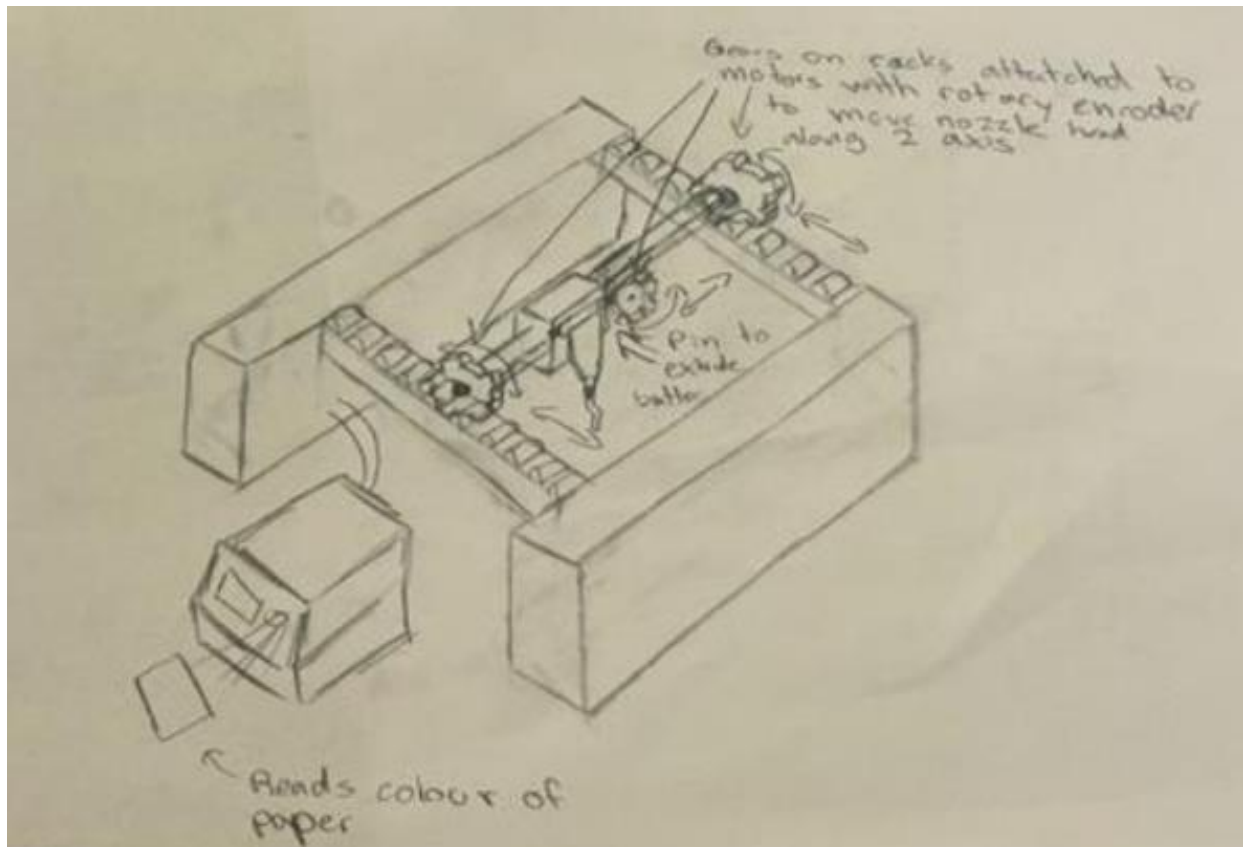


Figure 1: Conceptual design #1

The second conceptual design was centred upon a wheel and rail-based gantry system. The extrusion mechanism contained a funnel filled with batter with a motorized stopper to control the flow of batter. The EV3 brick is used to receive user input via its directional pad buttons. The use of the EV3 brick largely improves the user experience by supporting users with on-screen instructional guidance. Furthermore, the EV3 brick's variety of unique buttons and its built-in screen allows for more complex software design, that can result in more intuitive interaction and expanded functionality.

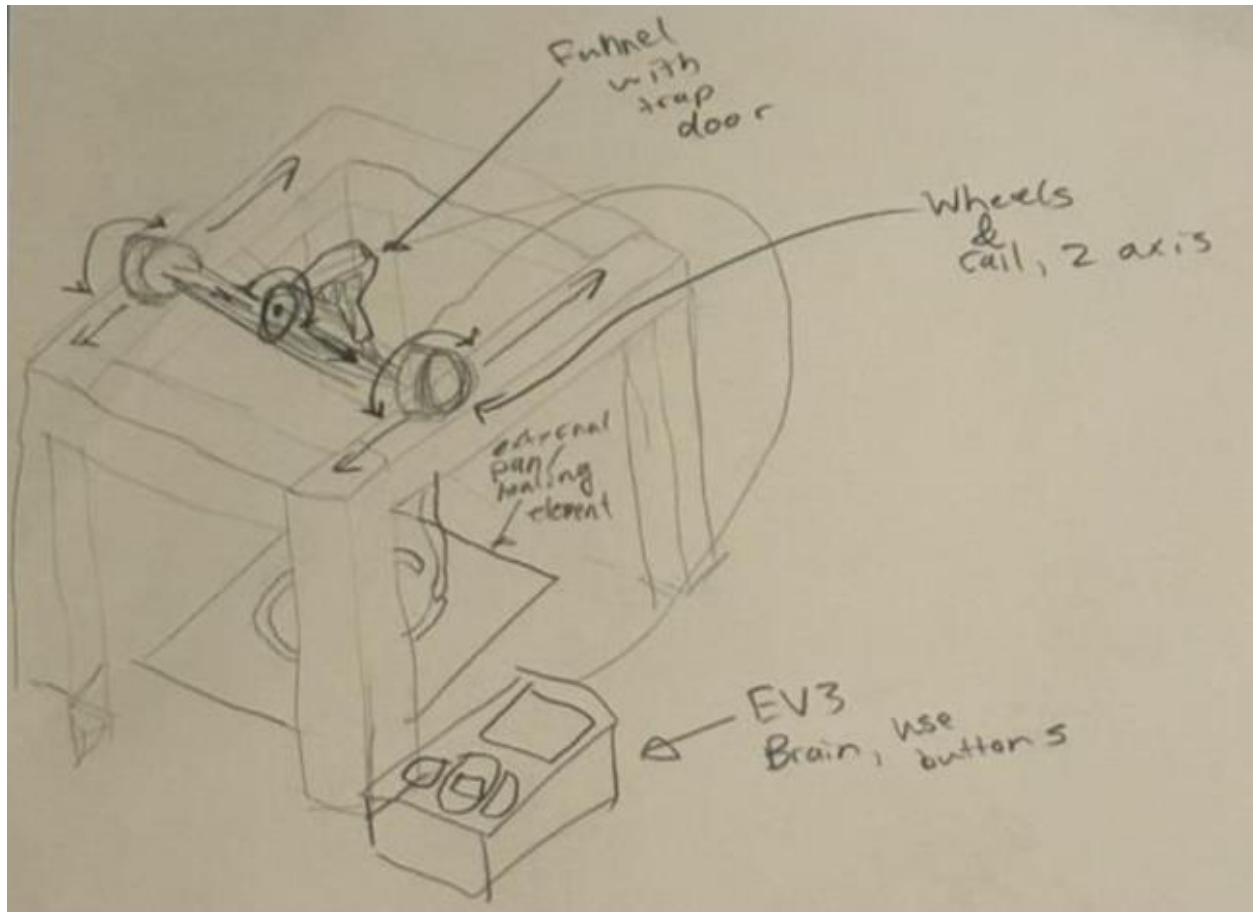


Figure 2: Conceptual design #2

The third conceptual design is very different from the previous conceptual designs. It features a robotic car that drives on a cooking surface, printing by extruding a trail of batter. The car would be entirely self contained, holding all its components on top of it. The extrusion mechanism uses a syringe that is pushed on by a rack and pinion. Like conceptual design #2, the EV3 brick is used to receive input and provide instructional guidance.

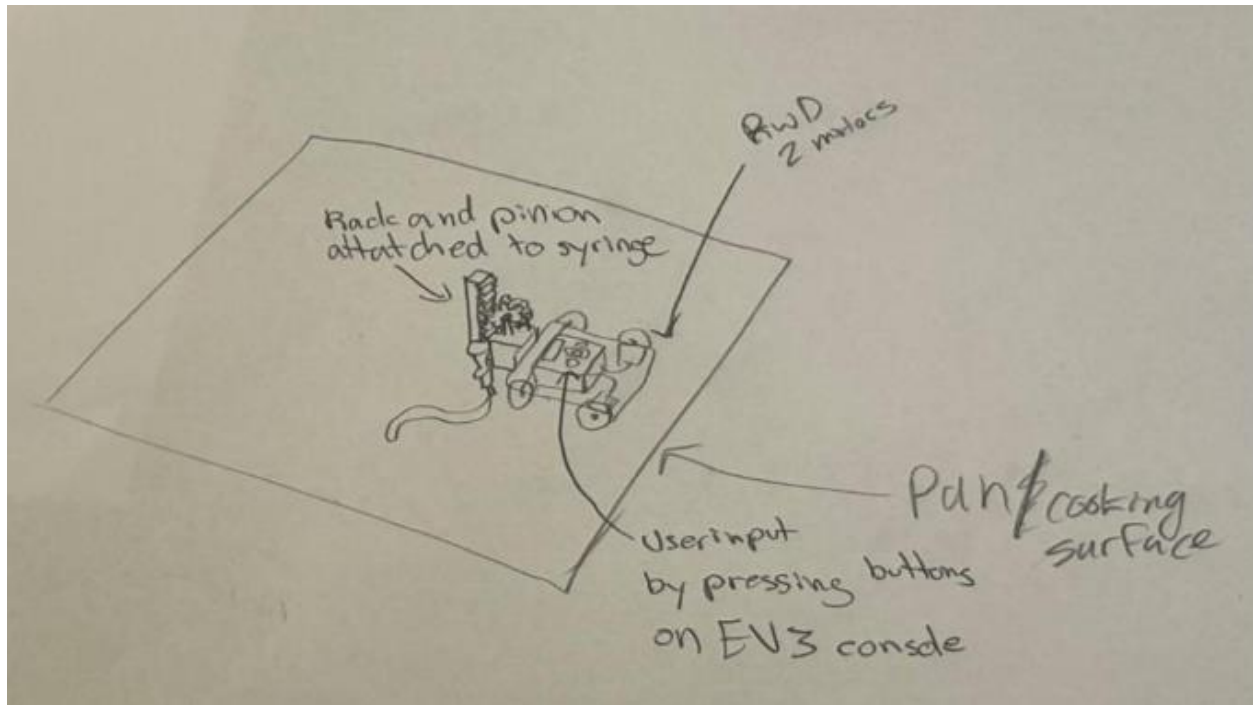


Figure 3: Conceptual design #3

2.2. Decision Making Matrix

A decision-making matrix is created where each conceptual design is evaluated relative to a selected datum design. The designs are evaluated in 5 categories to determine a “best” design. Table 3 contains the decision-making matrix used to evaluate all three conceptual designs; conceptual design #1 was selected as the datum design.

Criteria	Concept #1 (Datum) Rack and Pinion	Concept #2 Wheels and Rails	Concept #3 Free Roam Car
Accuracy	0	0	-1
Speed	0	0	+1
User Friendliness	0	+1	+1
Manufacturability	0	0	-1
Maintenance	0	0	-1

Total	0	+1	-1
-------	---	----	----

Table 3 Decision making matrix

Conceptual design #1, serving as the datum design, is given a score of 0 for every category, resulting in a total score of 0. Conceptual design #2 is given a similar score of 0 for every category except friendliness. The similar scores are a result of both designs using very similar gantry systems. The main difference between the designs is the user input the EV3 brick in conceptual design #2 excels due to its capacity for more complex software design and instructional support.

In contrast, conceptual design #3 demonstrates both strengths and weaknesses relative to the datum. In terms of speed, directly powering the wheels would likely result in faster and more efficient movement, particularly when printing curves. The user friendliness scores well because of its use of the EV3 brick's screen and buttons. However, despite its high speed, the car design would suffer from being inaccurate due to the limitations of the EV3 motors. Furthermore, while the gantry-based designs operate along independently controlled axes, the car design relies on movement relative to the position of the vehicle, introducing additional complexity. Lastly, the self-contained nature of the car means all the components will be squished onto a small frame making it substantially harder to both put together and maintain relative to a larger more modular gantry system.

Using the decision-making matrix to compare the three conceptual designs derived from the morphological chart, it was determined that conceptual design #2 was the most effective design.

2.3. Final Conceptual Design

Despite results of the decision-making matrix, the final conceptual design still ended up changing significantly for several reasons. A large flaw with the decision-making matrix was that it evaluated the system instead of evaluating individual functions. Evaluating individual functions is especially important when determining a conceptual design for a pancake printer. This is due to how independent each mechanism and their functions are. Each mechanism simply needs to convert some input into some output. For example, the sensors take physical user input and triggers an event in the code as an output. Similarly, the gantry and extrusion systems take some event triggered in the code and power in the motors, moving the nozzle and extruding batter.

The conceptual design was reevaluated one last time considering each function individually and choosing the best mechanisms individually. The resulting and final conceptual design utilized a wheel and rail gantry system, a rack and pinion batter extrusion system, and a combination of EV3 brick buttons, colour, and touch sensors for a user interface.

3. Mechanical Design

3.1. Layout

The functionality of the robot directly depends on the strategic arrangement of various components including sensors, motors, and frame components. The final layout, as shown in Figure 4, prioritizes functionality, manufacturability, and ease of use. The mechanical design can be broken down into three main components: the frame, the drive train, and the EV3 brick and sensors. Figure 4 shows a SolidWorks assembly of the final design of the robot.

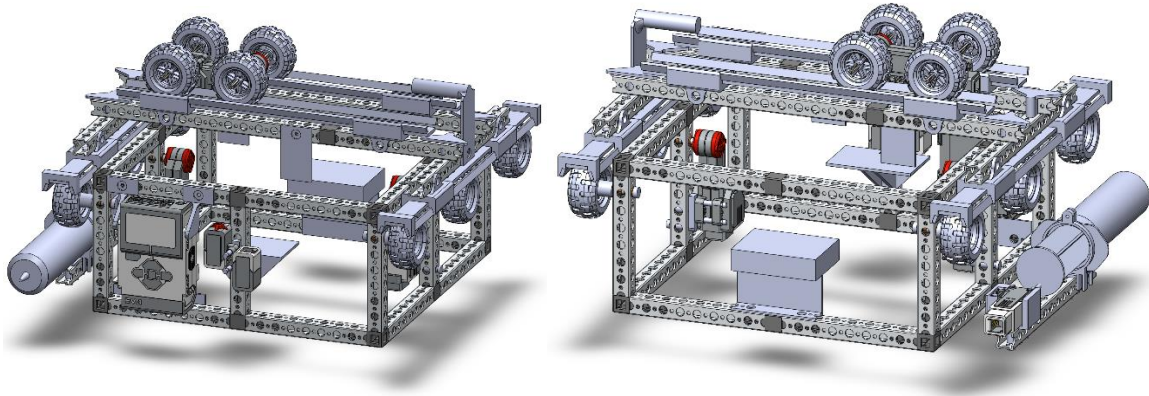


Figure 4: SolidWorks Visualization of Final Design (Front and back view)

3.1.1. Frame Design

The robot's overall design is centered around its gantry rail and the frame that it is built off. Pictured below in Figure 5, is the final design for the frame.

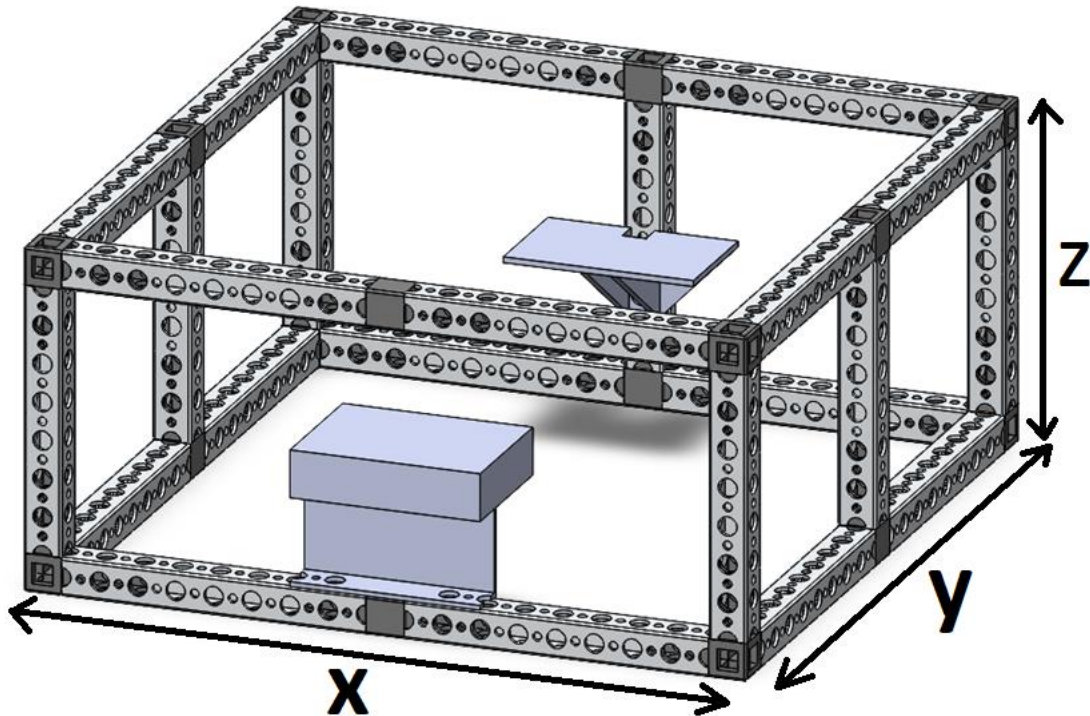


Figure 5: SolidWorks Assembly of Final Designed Frame

The frame is comprised of seven 6-hole beams, sixteen 7-hole beams, six T-Connectors, eight 3-Way Connectors, and two straight Connectors. The frame also includes a platform consisting of two screwed on pieces which allows pans to sit on. The Tetrix beams running along the Z axis were initially 12-hole beams instead of the now 6-hole beams. The frame height was decreased as the initial height was found to be excessive, hindering the ease of storing the robot. 6-hole beams were specifically chosen as they were the smallest available beams that could be used, while leaving enough space for the EV3 brick and motors to be mounted. The back of the frame (shown as the front in Figure 5) lacks a midline support beam that the three other faces have. This configuration provides the frame with structural support while maintaining its functionality.

The gantry rails shown in Figure 6 provides rails along the X and Y axes, allowing for nozzle movement along two axes.

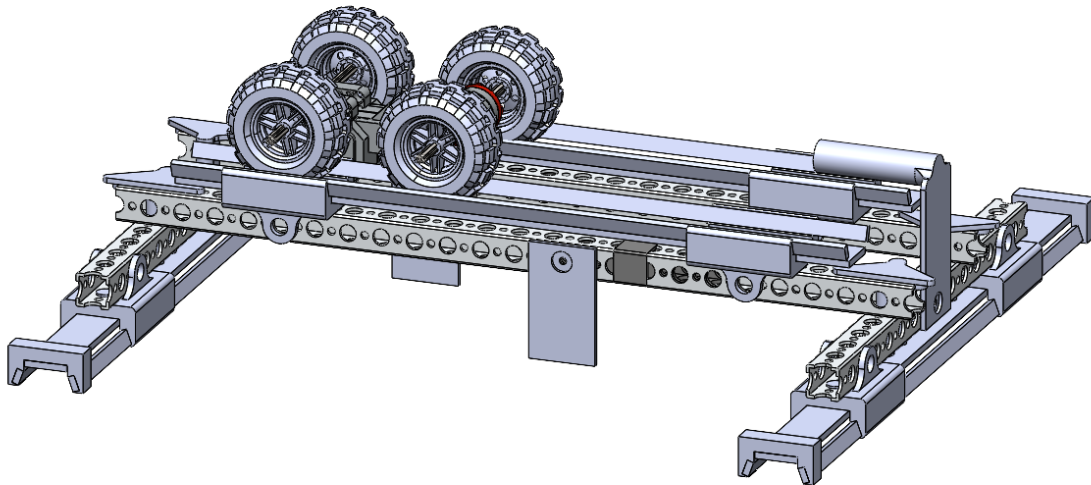


Figure 6: Dimetric View SolidWorks Assembly of Fully Designed Nozzle Extrusion Cart Sitting on X-Direction Rail Gantry and Y-Direction Rail

The rail consists of four 14-hole beams, two 7-hole beams, and two straight connectors. The beams that weren't connected using straight connectors were fastened together using a pin shown in Figure 7 that was locked in place with a C-clip. The pin was designed to sit partially flush with a beam to prevent rotation of the two beams during operation.

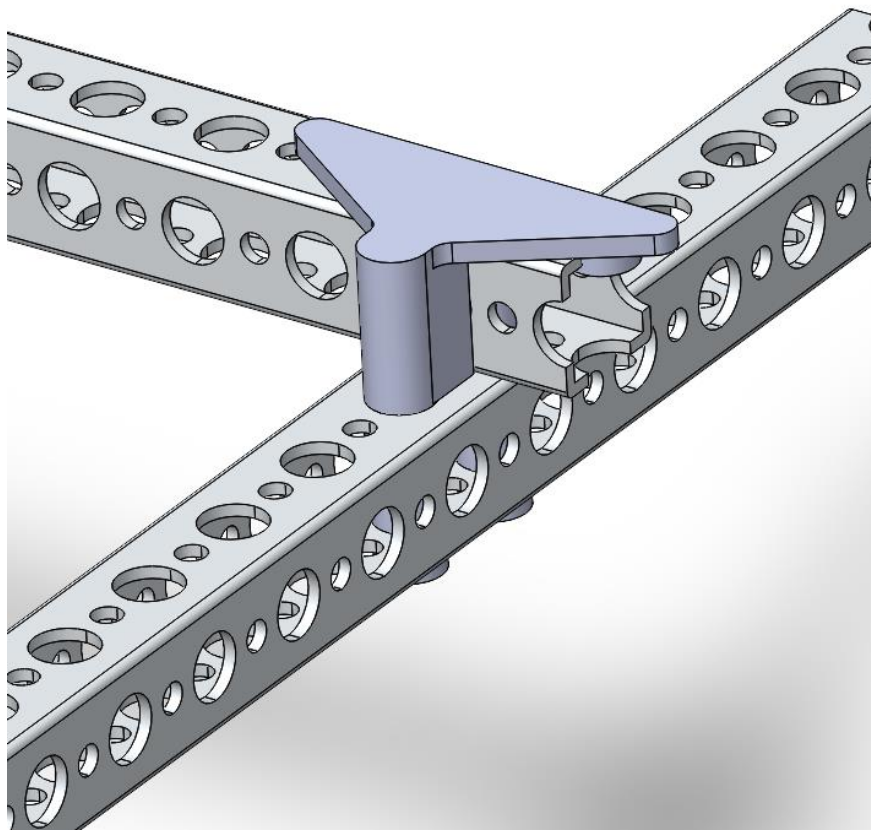


Figure 7: Pin Used to Mount X-Axis Rail to Y-Axis Rail

The rail itself is comprised of two main components: the rectangular strips the wheels sit on, and the bracket mounting these strips onto the beam. Each consisted of a center strip working as the main surface the wheel sits on, and two strips on the side acting as guide rails. Through iterative testing, an optimal angle between the three strips was determined to maintain minimal

friction while preventing derailing. Each mount was fastened onto the beam using a pin over a friction fit as it would more easily be disassembled and reassembled.

While testing, it was observed that the ends of the Y rail was found to begin warping when the wheels sat on them, as the Y rail's strips extend far past their mounts. To fix this issue, attachments that matched the profile of the brackets were attached onto the endpoints of the Y rail's strips to reduce warping.

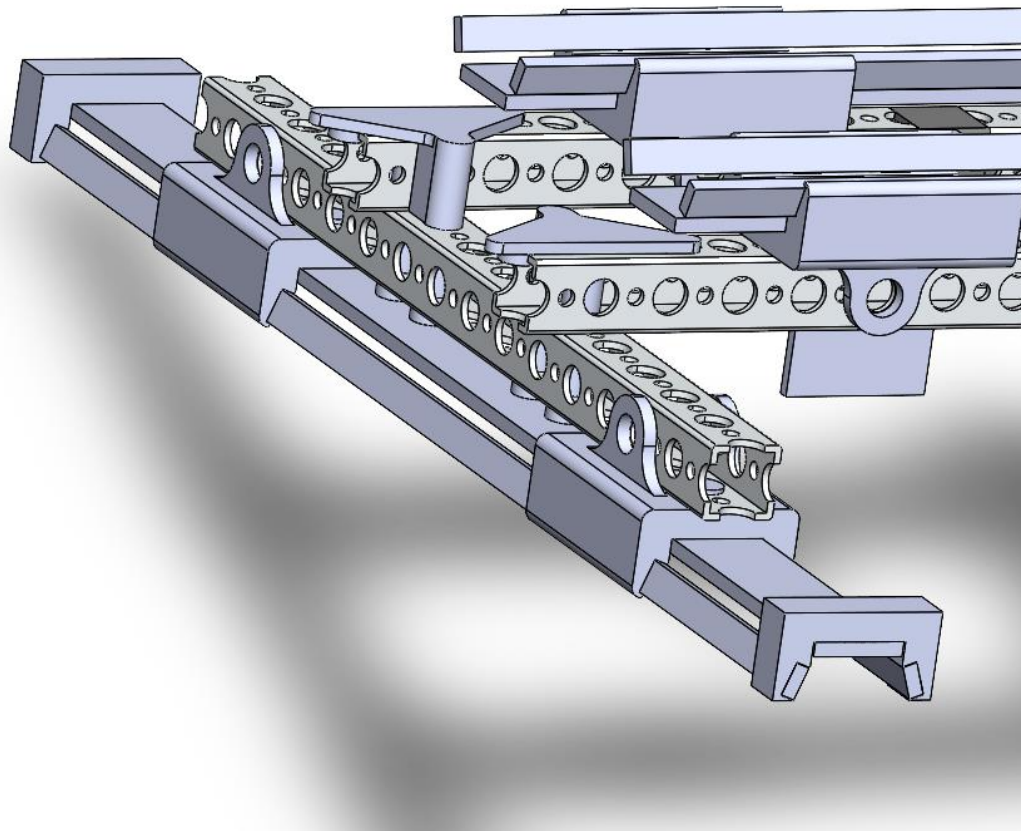


Figure 8: SolidWorks Assembly of Central Rail Mounting Brackets

Attached on the gantry rails were bumpers which would act as physical stoppers to prevent the nozzle from travelling past its boundaries along both axes. As shown in Figure 9, these bumpers were screwed on to the gantry rails, the X axis bumpers contacting the cart, and

the Y axis bumpers contacting the frame. Although software was implemented to prevent the nozzle from travelling past its boundaries, it was decided that physical barriers should be used in the case of a software malfunction. The physical barriers also serve a dual purpose, operating as a zeroing function, by having robot driving into two bumpers, which would locate it to a precise location.

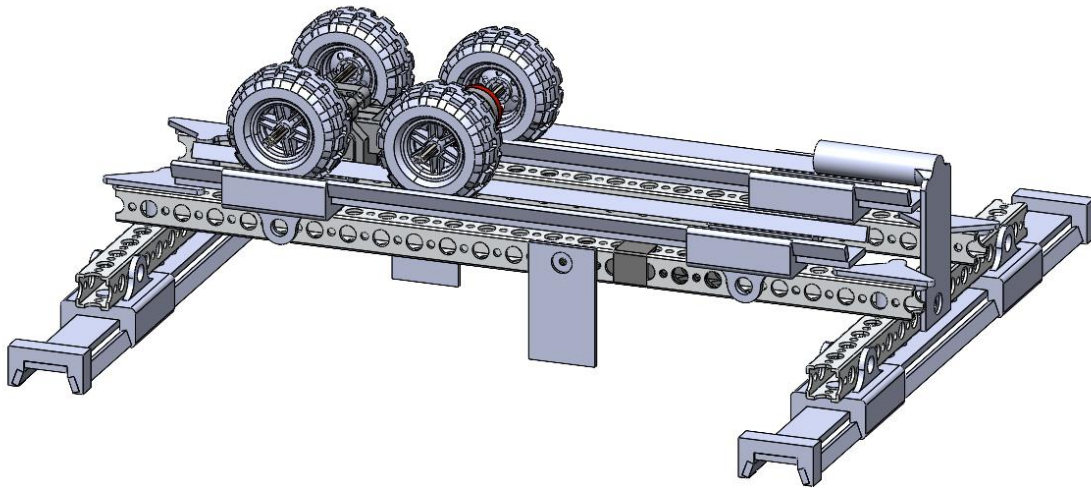


Figure 9: Dimetric View of Nozzle Extrusion Cart XY Rail System in SolidWorks Assembly

3.1.2. Extrusion and Drive Train

The robot's powered components consist of three parts: The extrusion system, the Y-axis movement system, and the X-axis movement system.

Figure 10 shows the extrusion system, designed to be detached from the main frame for easy syringe refilling. The system is mounted on a 12-hole Tetrix beam, with brackets securing a

low torque EV3 motor and syringe onto the beam. A lead screw was used to transfer the rotational force of a motor to a linear force, pushing the piston to extrude the batter. Initially, a laser-cut rack and pinion was considered and tested but replaced by a lead screw was due to the incompatibility of the rack and pinion. Rack and pinion systems are better suited for low-torque high-speed applications, whereas the extrusion system requires high torque, making the lead screw more appropriate.

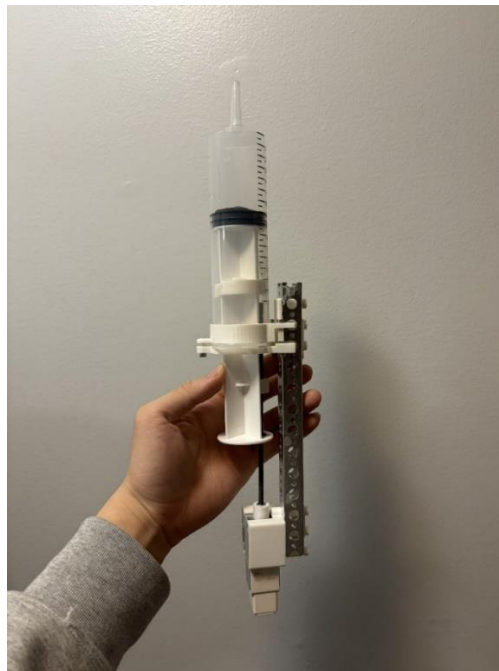


Figure 10: Batter Extrusion Mechanism

The high-torque environment of the extrusion system meant that heavy forces would be applied to its components. To operate reliably, the components in the extrusion system had to be designed to withstand operational forces with minimal deflection. Minimizing deflection was critical as testing the extrusion system revealed that small misalignments caused operational forces resulted in a decrease in force output on the syringe. This occurred because the

misalignment caused to apply force at an angle, instead of directly parallel to the piston's movement.

To reduce misalignments caused by operational forces, the components involved in the extrusion mechanism were built to withstand greater forces. Improvements included tighter friction fits, bolted components, and more fastening points. The bracket fastening the low-torque motor onto the beam was reinforced with friction fitted walls and four friction-fit pins to ensure a snug fit and minimize deflection. Figure 11 shows a cross-sectional view of the motor attached to the bracket.

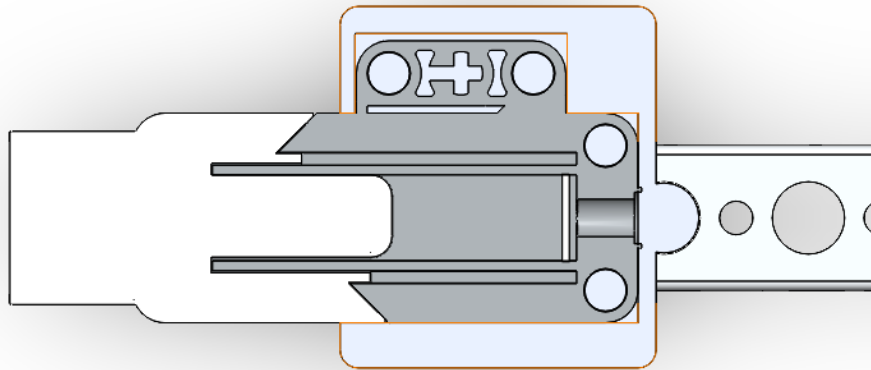


Figure 11: Cross Sectional View of Extrusion Motor Bracket

The bracket mounting the syringe onto the beam consists of three parts that are bolted together to ensure a secure fit. To enhance stability, the bracket is mounted onto the pin using three pins instead of the standard two pins typically used to mount other components into the robot.

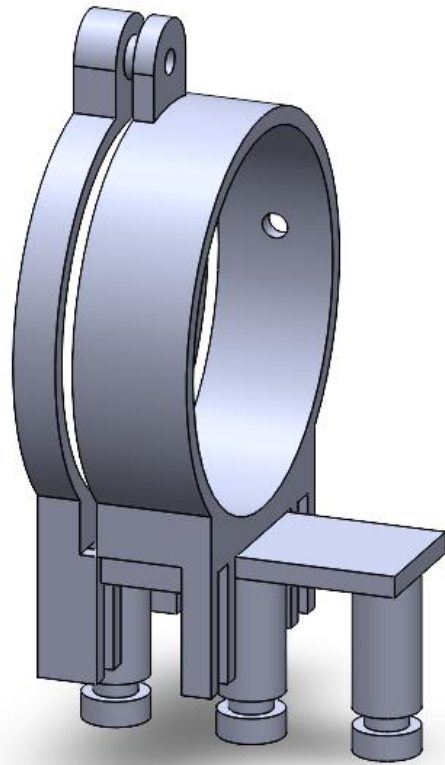


Figure 12: Bracket Mounting Syringe to Beam

The Y axis drive system works by rolling two wheels found at opposite sides of the robot's frame to move the nozzle along the Y axis. In Figure 13, The wheels located at the four corners aren't powered while the other two are each powered by EV3 motors.

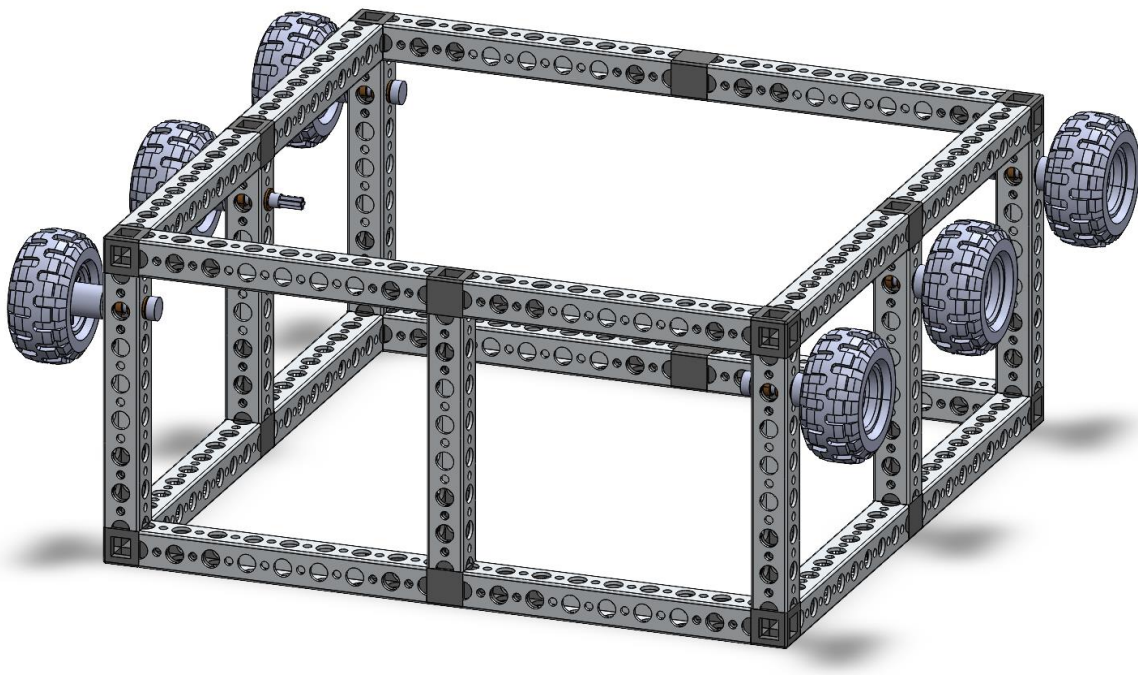


Figure 13: Frame With Y-Axis Wheels Mounted

The design of the axels is between the two mounts as basically identical, aside from the fact that one attaches to a motor. The wheel's axel fits through a Tetrix beam hole with brass bushings separating the axel from the holes. The axels have spacers which were determined by measuring the distance between the two Y-axis rails. The motor is mounted onto the frame using pins that are secured using C-clips.

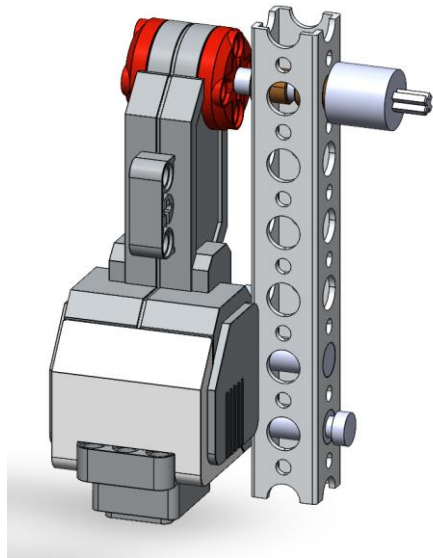


Figure 14: Y-Axis Motor Drive System in Assembly

The X-axis movement system consists of a single motor built as a cart. The cart drive also the axis with two of its wheels powered. Although having the cart balance on two wheels was a consideration, it was decided that four wheels would be used as it completely rids the worry of balance.



Figure 15: Top View of Assembled Robot Showing Cart

3.1.3. EV3 Brick and Sensors

Figure 16 shows the front of the frame where all the EV3 sensors and the brick are mounted. The EV3 brick is connected to the frame under the extrusion cart at the origin of the printing area.

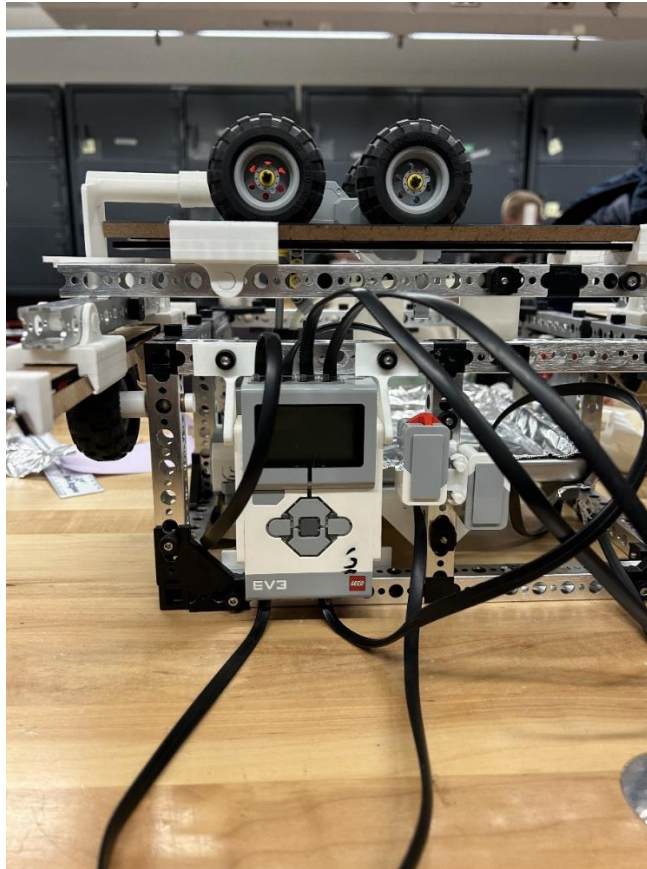


Figure 16: Front Facing View of Final Design with EV3 Brick, Touch Sensor, and Colour
Sensor

The sensors involved in the robot was the EV3 brick, touch sensor, colour sensor, and motor encoders. The integrated buttons found on the brick were used for controlling the menu and the robot in manual mode. The menu controls included entering manual mode and

confirming whether actions should be carried out. The reason the integrated button was used for confirmation messages was the buttons would be right below the console where the user would read confirmation messages. The integrated buttons were used as controls for manual mode as the orientation of the arrow buttons would make controlling the nozzle intuitive for the user (left button travels left, right button travels right).

The EV3 brick is mounted onto the frame using two brackets as shown in Figure 17. The designs of the two brackets are mirrors of one another and work by screwing onto the frame and fitting into the pins found at the side of the EV3 brick. A problem that was encountered while testing the mount was that the EV3 brick would bend towards the inside of the frame when buttons were pressed due to how thin parts of the bracket were. To prevent the brick from bending, a physical barrier was mounted to the bottom of the frame. The brick was mounted upright as it would be easier for users to read the console outputs and press buttons. The colour sensor and touch sensor were mounted using the same mount that was used for the motor onto the frame. Coincidentally, it worked as a mount for the two sensors which eliminated the hassle that would come from having to design a unique mount for the sensors. The two sensors were mounted close to one another for an easier user experience as the use of the colour sensor required the simultaneous use of the touch sensor. The brick and sensors were intentionally placed close to each other as having all the interactive components close to each other makes it easier for the user.

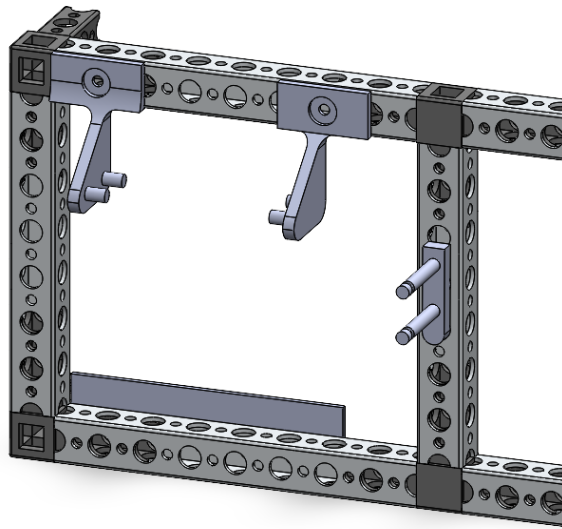


Figure 17: Brackets Mounting the Brick And Sensors Onto the Frame

3.1.4. Fabrication

Figure 18 and Figure 19 show the total collection of 3D printed and laser cut components used in the final assembly of the robot. Most of the components that were manufactured were fabricated using 3D printing. This is because most of the components included 3D geometries which laser cutting couldn't produce. An issue that occurred around 3D printing that had to be resolved was the tolerance. Based on the tightness of the fit required and the type of material the 3D printed part was fitting with, tolerances had to be adjusted. The 3D printed parts were built to fit with EV3 parts, Tetrrix parts, or other 3D printed parts. Friction fits and loose fits were required depending on the application. Each of these applications required testing to find the optimal tolerance needed to correctly fit the pieces together. The main reason that PLA was used for all the parts was that it was the most widely available 3D printing material, and the material

properties of strong materials such as PETG weren't required for the applications of the 3D printed parts found in the robot.

The parts that were laser cut were the rails running along the X and Y axis. Using 1/8-inch wood, rectangular strips of wood were cut for the wheels to run along. It was decided to laser cut these parts as firstly, they wouldn't fit on a print bed. 3D printing the strips would have required segmentizing the print into multiple parts and finding a way to attach them together. Additionally, it would be much more efficient in terms of time and cost to laser cut the strips instead of 3D printing them.

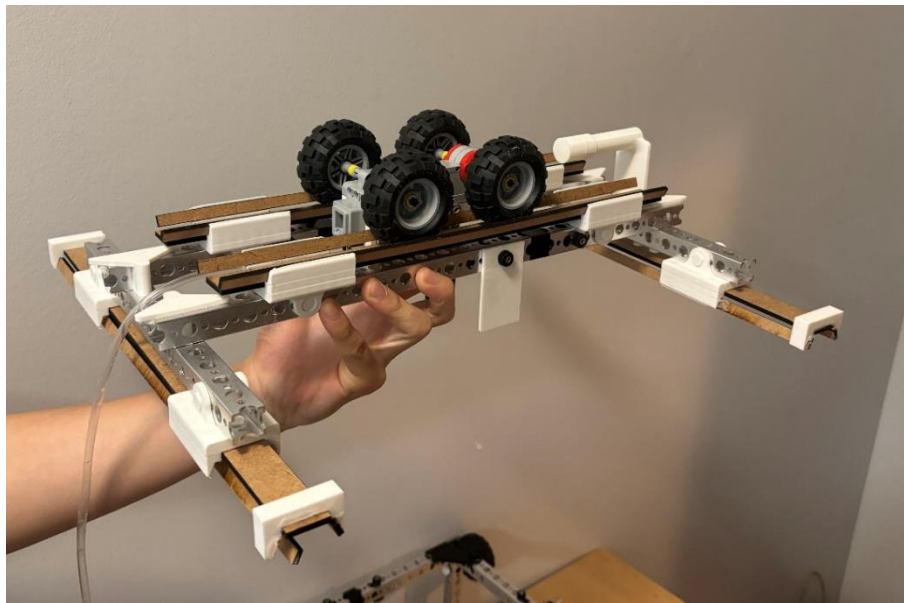


Figure 18: Laser Cut Rails shown in Full Rail Assembly

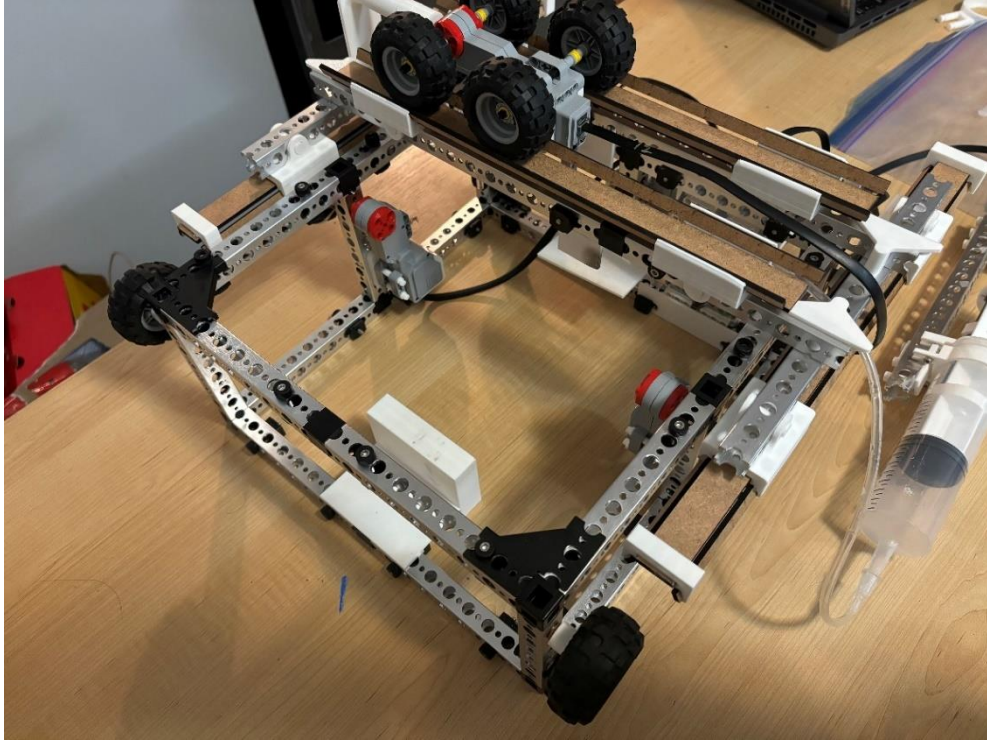


Figure 19: Printing Area Including Printing Tray Supports and Connected Syringe

4. Verification of Design

4.1. Verification of Specification

There are seven specifications or criteria that the final design must meet as outlined in **Table 1**. The verification of each specification will be demonstrated as each comment outlines during the presentation of the final robot. Each specification will be assessed if they fully meet, partially meet, or do not meet expectations.

4.1.1. Difficulty of Operation

The first specification in the table is difficulty of operation, which was verified via demonstration with a student that was uninvolved with the project. The student was briefed on

the basic operation of both the manual mode and colour detection mode of the robot. Through this demonstration, the individual successfully printed using both modes, confirming that this specification was met.

4.1.2. Consecutive Uses

The second specification outlined in the table is consecutive uses, which was verified using a test during the live demonstration of the robot. This specification is successfully met if the robot can execute three or more prints without failing any major issues. During the live demonstration, the robot printed all three preprogrammed shapes without any major issues. Thus, this specification was successfully verified.

4.1.3. Dimensions

The third specification outlined in the table the dimensions of the robot itself. The to meet the specification, the robot must measure less than 50 cm in length and width and less than 40 cm in height when in its stored position. It was verified by directly measuring each dimension using a ruler during the live demonstration. The resulting dimensions were well under the specified values, measuring in at around 40 cm in length and width, and 25 cm in height. As a result, this specification was successfully verified.

4.1.4. Mass

The fourth specification outlined was regarding the overall mass of the robot and was successfully met if the mass was at less than or equal to 6 kg. The specification was tested by directly weighing the robot on a scale, coming in around 2.2 kg. If the syringe is full and the tray is included in the measurement, it all weighs around 2.5 kg. Since this is significantly below the 6 kg limit, this specification was successfully verified.

4.1.5. Dispensable Batter

The fifth specification addresses the amount of batter the robot can dispense before requiring a refill. The specification was tested by finding the change in batter between the initial amount of batter and the remaining amount when a refill was needed. If it was greater than 75 mL it was verified. Prior to the live demonstration, the syringe was filled up to the 90 mL marking and by the end of the demonstration, the syringe was empty meaning that it had dispensed around 90 mL of batter. This test confirmed that the robot exceeded the minimum requirement of 75 mL, successfully verifying this specification.

4.1.6. Printable Area

The sixth specification was to have at least 225 cm² of the printable area. This was verified by measuring the maximum lateral and longitudinal displacement of the nozzle using a ruler. Testing revealed that the nozzle was able to travel 19 cm in laterally and 24 cm in longitudinally, resulting in a total printing area of approximately 456 cm². As this exceeds the specified minimum of 225 cm², the specification was successfully verified.

4.1.7. Deformation of Fabricated Parts

The seventh and final specification evaluates whether the fabricated parts of the robot experience deformation due to factors such as load or heat from the motors. This was assessed through visual examination conducted multiple times prior to and during the live demonstration. No visible deformation was observed after the demonstrations, confirming that this specification was successfully met.

4.2 Overall Assessment

Overall, the robot successfully met all the outlined engineering specifications. However, while all the specifications were met, the robot was still far from perfect. The specifications did not account for certain aspects such as the extrusion mechanism's ability to control the flow of batter and ease of maintenance. Specifically, when the extrusion mechanism was toggled off, the batter would often continue to drip. Or if the batter was too viscous, the adhesive forces between the batter and the tubing would make the robot completely unable to extrude. Additionally, robust design of the extrusion mechanism made it extremely difficult and messy to refill the syringe. Despite these challenges, most issues were confined to the extrusion mechanism. In contrast, the gantry system and user interface performed exceptionally well, functioning almost exactly as intended and demonstrating a high level of success. Issues with the gantry system and user interface were directly tied to the limitations of the Lego EV3 motors and sensors.

5.0. Software Design and Implementation

5.1. Description of Software

Software design is a critical phase in the development of any product, as it lays the foundation for how the system will function and how users will interact with it. Effective software design not only limits the amount of potential software errors that may occur during runtime but also allows for hardware limitations to be overcome through easy to implement solutions.

5.1.1. Flow Charts

In figure 20, the entire process of the code behind the pancake printing robot is outlined. The code is primarily broken up into two distinct sections, the first being the colour detection and the second being the manual mode. These two modes are separated based off user input allowing a high level of control for the user.

The first mode is entered with the touch sensor. When the touch sensor is activated the color sensor scans for either red, blue, black, or white. Each individual color is mapped to a different function and a confirmation message prompts the user to verify the choice. Scanning red, blue, and black will print a square, triangle, or spiral respectively and scanning white will prompt the user asking if they want to exit the program. If the user shows a non aforementioned color then it will exit the mode and restart the program again asking for the initial user input.

The second mode that can be entered is manual mode. Manual mode is entered by touching the EV3 brick's keypad. This mode allows the user to create their own designs by controlling the movement of the nozzle and the extrusion of the batter. When in manual mode clicking the touch sensor will reset the nozzle position and restart the program.

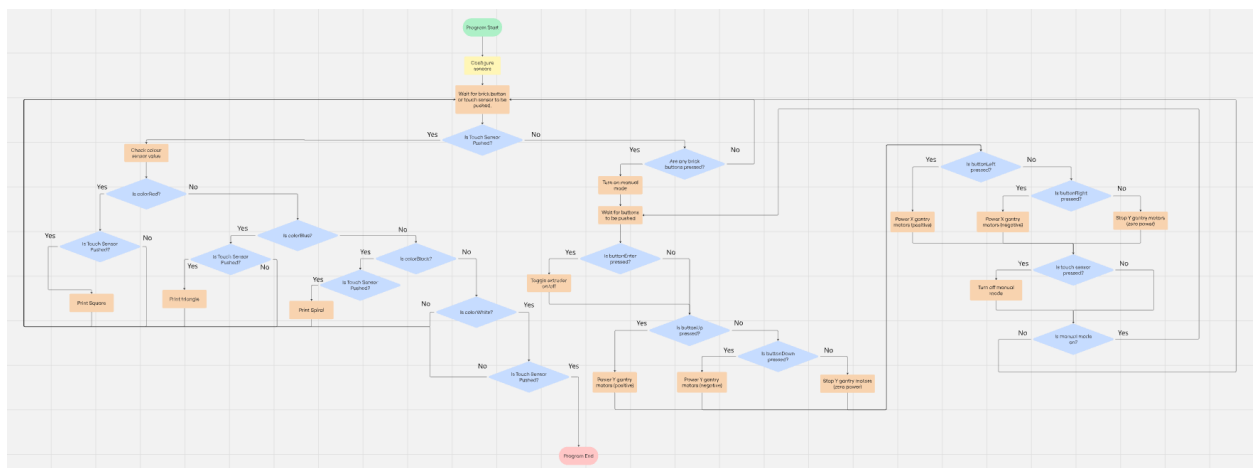


Figure 20: Diagram of Full Programming Flow Chart

5.1.2. Task List

The task list used during the live demonstration of the robot is as follows:

- ☐ Starting Up
 - ☐ Prompts Users for Input Before Anything
 - ☐ Brick Buttons to Enter Manual Mode
 - ☐ Touch Sensor Initiates Colour Based Shape Printing
- ☐ Operation
 - ☐ Manual Mode
 - ☐ Movements Based Off Directional Buttons
 - ☐ Enter Toggles Extruder On / Off
 - ☐ Cannot Manually Drive Past Virtual Limits
 - ☐ Colour Detection Mode
 - ☐ Prints Respective Shape or Exits Programs Based Off Detected Colour
 - ☐ If Red, Prints Square
 - ☐ If Blue, Prints Triangle
 - ☐ If Black, Prints Spiral
 - ☐ If White, Exits Program
- ☐ Interactions with Environment
 - ☐ Prompts to Screen Confirming Correct Colour Detected
 - ☐ Zeroing / Nozzle Resetting to Virtual Coordinates (0,0) Intermittently

- ☐ Physical and Virtual Limitations to Nozzle Movement in Case of External Interface
- ☐ Shutting Down
 - ☐ Initiates Shut Down Procedure When White is Detected
 - ☐ Prompts User to Confirm Shutdown
 - ☐ Zeroes the Nozzle Before Shutdown
 - ☐ Displays “Thank You!” Message

The task list is broken down into 4 main blocks to highlight the main stages of the program these being: start up, operation, interactions with the environment, and shutting down. These tasks were chosen for the demo as it highlights every way that a user or the environment may interact with the robot in a reasonable fashion.

One change to the task list is the replacement of the face for the spiral print. This change is the result of limitations with the control of the extrusion rate. Stopping the extrusion was inconsistent and would lead to extremely messy prints. If a shape required the constant starting and stopping of batter extrusion this inconsistency would become more prominent. The second change made to the task list was the addition of virtual limits. In manual mode the user is able to drive the gantry or cart into the bumpers indefinitely however when displaying the coordinates of the current nozzle position the values would be offset as the wheels would still turn updating the motor encoders while its physical position would stay the same. To prevent this from happening virtual bumpers were added making it so once the x or y position reached certain values movement in that direction would stop.

Following the above listed task list was a separate checklist for the seven engineering specifications previously outlined in **Table 1** as well as their respective verifications.

☐ Difficulty of Operation ☐ Demonstration (Live Peer Use) ☐ Consecutive Uses ☐ Test (Run Program Several Times) ☐ Dimensions ☐ Length ☐ Test (Measure w/ Ruler) ☐ Width ☐ Test (Measure w/ Ruler) ☐ Height ☐ Test (Measure w/ Ruler) ☐ Mass ☐ Test (Weigh on scale) ☐ Volume Dispensed ☐ Test (Measure w/ Syringe Markings) ☐ Printable Area ☐ Test (Measure w/ Ruler) ☐ Components Resist Deformation ☐ Examination
--

5.1.3. Functions

5.1.3.a float getY() – Marwan Z.

The getY() function is a simple function which takes the motor encoders value for the y-axis gantry and returns its distance away from the origin in centimeters.

5.1.3.b float getX() – Marwan Z.

The getX() function is a simple function which takes the motor encoders value for the x-axis cart and returns its distance away from the origin in centimeters.

5.1.3.c void extrude(bool on) – Nathan M.

Based on the parameter this function toggles the motor which extrudes the batter on or off.

5.1.3.d void move_Both(float x, float y, float ex_minX, float ex_maxX, float ex_minY, float ex_maxY) – Calim KW.

This function is the building block of the code as it acts as the control for both the movement and extrusion. The x and y parameters are the desired final position of the nozzle and based off its current position the direction at which each motor must move is determined. The last four parameters are used to determine at which positions extrusion should be enabled. If the current x position is found to be between the ex_minX and ex_maxX values and the current y position is found to be between the ex_minY and ex_maxY values then the extrusion will be toggled on.

5.1.3.e void prepare(int x, int y) – Calim KW.

The prepare function works as a special case of the move_Both function by preparing the nozzle at a certain point based off the parameters provided. In addition to moving the nozzle to that point it also toggles the extrude for a little bit to ensure that the batter is at the tip of the nozzle ready for a print to begin.

5.1.3.f void reset() – Calim KW.

The reset function moves the nozzle back to (0,0) however it also turns on the motors to drive them into the bumpers to ensure that it is in the very corner. It then sets this as the new (0,0) through the motor encoders. This function is vital for the precision in the robot as cords will often get caught during prints and offset the position by little bits resulting in large positional errors over time.

5.1.3.g void square() – Calim KW.

The square function uses a combination of the aforementioned functions to extrude a square of batter onto the plate. The function achieves this by first using the prepare function to get it ready in its first position then using a series of the move_Both function to draw out a square using the coordinates of the corners. After the square is finished being extruded it returns to the origin using the reset function

5.1.3.h void triangle() – Calim KW.

This function similar to the square as it is a combination of the prepare, move_Both and reset functions. The prepare function is used to set the initial nozzle position and then three

move_ Both functions are executed to create the triangle geometry. After the print is complete the reset function is used to return to the origin.

5.1.3.i void spiral() – Dylan L.

The spiral function works the same way as the square and triangle function defining coordinates to move to and upon finishing the robot returns to the origin using the reset function.

5.1.3.j void manual(bool & on) – Nathan M.

The manual function controls the position and extrusion of the nozzle through user inputs. The on parameter is used to toggle the extrusion each time the enter key is pressed by switching its boolean value. This function continuously checks what keys are being pressed and moves the nozzle accordingly allowing for a reactive user experience. The manual movement of the nozzle has additionally conditions to prevent the wheels from rotating when the robot is stuck against a bumper to ensure an accurate relation between the virtual position and the physical position.

5.1.3.k void colorShape(bool & system) – Dylan L.

Based on what the color sensor detects it will draw one of the three shapes or exit the program. The function prompts for confirmation to ensure that the correct color was scanned in order to overcome the inaccuracy of the color sensor. Additionally this function is what allows the shutdown of the program through the system parameter as if the color white is scanned it will switch that variable to false.

5.1.3.1 int confirm(string message) – Nathan M.

This function prompts the user to confirm that the color they scanned will print the shape that they want. If the user confirms the shape it returns a one and if they don't it returns a zero. the message parameter allows it to act as a confirmation for any of the three shapes.

5.1.3.m task main() – All

The main function is in charge of all the setup of the sensors and the initialization of the operations. The main function prompts the user to either enter manual or the color based autonomous mode and from there the functions outlined above control the rest whether it be user input or autonomous decision making.

5.1.3.n Function Design

The idea of the program is to be built off a couple key functions and then through abstraction develop more complex functionality for the robot. This idea is highlighted through the use of the move_Both function throughout the different shapes. In figure 21 the lengthy code that drives the movement and extrusion of the nozzle is shown. Having all that code wrapped into a single function allows for each shape function to be made relatively simple only requiring a couple lines of code as shown in figure 22. For example the square function makes use of the prepare, move_Both, and reset function and while all of those are instances of the code in figure 420 being used, the individual functions that are made to encapsulate them help aid in readability and shortens the amount of code that needs to be written by a substantial amount. Additionally, the simplification of the code allows for the easy addition of new shapes on the fly which benefits not only the manufacturers but also the users if the project is released as open-source as even users with little coding experience can easily work with the codebase.

```

void move_Both(float x, float y, float ex_minX, float ex_maxX, float ex_minY, float ex_maxY)
{
    while(abs(x-getX()) > TOLERANCE || abs(y-getY()) > TOLERANCE)
    {
        if(ex_minX == ex_maxX)
        {
            ex_minX = BED_MIN; ex_maxX = BED_MAX;
        }
        if(ex_minY == ex_maxY)
        {
            ex_minY = BED_MIN; ex_maxY = BED_MAX;
        }
        int directionX = 0, directionY = 0;
        if(x-getX() > 0)
        {
            directionX = 1;
        }
        else
        {
            directionX = -1;
        }
        if(y-getY() > 0)
        {
            directionY = 1;
        }
        else
        {
            directionY = -1;
        }
        //if it is "close enough" to the desired x or y position it stops moving in that direction
        if(abs(x-getX()) < TOLERANCE)
        {
            directionX = 0;
        }
        if(abs(y-getY()) < TOLERANCE)
        {
            directionY = 0;
        }
        motor[motorC] = directionX * SPEED_X;
        motor[motorA] = directionY * SPEED_Y;
        motor[motorB] = directionY * SPEED_Y;
        //if it is within the extrusion bounds it toggles the extrusion.
        if(getX() >= ex_minX && getX() <= ex_maxX && getY() >= ex_minY && getY() <= ex_maxY)
        {
            extrude(true);
        }
        else extrude(false);
    }
    //safeguards, ensures that movement and extrusion is completely stopped before moving on from function
    motor[motorA] = 0;
    motor[motorB] = 0;
    motor[motorC] = 0;
    extrude(false);
    wait1Msec(100);
}

```

Figure 21: Code of Function Responsible for Moving Nozzle Cart

```

//square function, uses four corner points and draws lines between them
void square()
{
    prepare(4,2);
    move_Both(4, 16, 4, 4, 2, 16);
    move_Both(18, 16, 4, 18, 16, 16);
    move_Both(18, 2, 18, 18, 2, 16);
    move_Both(4, 2, 4, 18, 2, 2);
    reset();
}

//triangle function maps out 3 points and drives between them
void triangle()
{
    prepare(4,2);
    move_Both(18, 16, 4, 18, 2, 16);
    move_Both(18, 2, 18, 18, 2, 16);
    move_Both(4, 2, 4, 18, 2, 2);
    reset();
}

void spiral()
{
    prepare(4, 2);
    move_Both(19, 2, 4, 19, 2, 2);
    move_Both(19, 15, 19, 19, 2, 15);
    move_Both(6, 15, 6, 19, 15, 15);
    move_Both(6, 6, 6, 6, 6, 15);
    move_Both(15, 6, 6, 15, 6, 6);
    move_Both(15, 12, 15, 15, 6, 12);
    move_Both(10, 12, 10, 15, 12, 12);
    move_Both(10, 9, 10, 10, 9, 12);
    reset();
}

```

Figure 22: Code of Colour-Based Shape Printing

Figure 23 and figure 24, show what and when the code prompts to the screen. These prompts are always meant for the benefit of the user, whether it be to give instruction or to confirm a selection, like what is shown in the figures. The prompts shown in figure 23, are executed at the beginning operation just after starting up. These prompts let the user know that the machine will enter a different print mode based on input, and how input is received, that being through the colour sensor and the EV3 brick buttons. The prompts shown in figure 24 are

used when the robot receives input from the colour sensor, and they ask the user whether the color the color sensor picked up was correct. This allows users to confirm whether the correct shape is being printed. After confirming, the robot then lets the user know that printing has begun and that hands should be kept at a safe distance from the printing space, prioritizing the experience of the user.

```
task main()
{
    configureAllSensors();
    //24x19 bed size
    bool system = true;
    //the main loop which enables the program to run/be broken out
    while(system)
    {
        displayString(3, "WAITING FOR INPUT");
        displayString(4, "KEYPAD FOR MANUAL");
        displayString(5, "TOUCH SENSOR FOR AUTOMATIC");
    }
}
```

Figure 23: Image of Main Function with User Instruction

```
int confirm(string message)
{
    displayString(3, "Printing %s", message);
    displayString(4, "Press enter to confirm");
    int num = 0;
    while(!getButtonPress(buttonAny))
    {}
    if(getButtonPress(buttonEnter))
    {
        num = 1;
        displayString(3, "Now Printing %s", message);
        displayString(4, "Please Keep hands away");
    }
}
```

Figure 24: Image of Confirm Function with User Message

5.1.4. Data and Trade-Offs

In any software project, effective organization of data is crucial to ensuring efficient processing, retrieval, and manipulation of functions. The data organization strategy primarily includes the use of variables and global constants, allowing for easier accessing, updating, and manipulating throughout the program. Majority of the variables are declared as the float type as they are able to generate a higher level of precision especially when dealing with irrational numbers like pi. For the same reason many of the functions return a float.

In terms of trade-offs and sacrifice, the most notable example experienced in this project came just prior to the day of live demonstration. The spiral function was originally meant to print out a face, but because of the inaccuracy of the syringe-based extrusion, printing the features of the face would result in a lower quality design that did not fit the sharpness of the other programmed shapes. The inaccuracy stems from the problem of batter dripping from the tube when it is not meant to extrude, which is what made the features of the face difficult in comparison to the other shapes as the act of stopping the extrusion often would not occur in time. The spiral shape was substituted instead for its long and continuous pattern, and for its constant extrusion compared to the face shape that required pauses in the middle printing.

5.2. Testing Program

Testing is a critical part of any software development process, as it is especially important to ensure that the code functions as intended and is free from errors. In this project, rigorous testing was implemented to verify the accuracy, reliability, and efficiency of the overall

code. Evaluations were conducted on each branch of code, testing their respective and shared functions.

For movement functions, precision is the key constraint that determines the successfulness of the function itself. The `move_Both` function was tested by measuring each axis of movement at a time next to a ruler to see how accurately the physical position reflected the virtual one. Through the tests the `move_Both` function was deemed to be accurate enough for use even without the use of a PID control system, something that was initially meant to be implemented.

The shape functions accuracy was determined by the accuracy of the `move_Both` functions however for the general shapes that were produced they were tested by attaching a marker to the cart and executing the functions. the markings left behind would provide an accurate representation for the path that was followed by the nozzle allowing for the tests to show whether it was the function providing inaccuracy to the print or the extrusion process of the batter.

Compared to the shape and movement functions, the rest of the functions are simpler to evaluate for effectiveness. The `extrude` function from the program can simply be calibrated by toggling the extrusion command on and off, measuring the displacement of the syringe plunger and adjusting the extrusion motor accordingly. The `shape colour` function can be tested using the EV3 brick's built in sensor testing feature and then comparing the results to the actual colour shown to the sensor. Lastly, the `manual` function can simply be tested by operating the directional button on the EV3 brick, ensuring that the nozzle movement corresponds with the user input. Other tests and testing of the overall program can be done by just simply playing with

the robot itself and trying random inputs over varying intervals of time, examining for any error or mistake in code and functions of the robot.

5.3. Programming Problems

The most significant programming problem that arose during this project was the timing of commands. At the start of prints there would often be a delay between the start of the movement and the extruded batter hitting the print plate. This delay was caused by multiple factors but the solution was rather simple. The problem can be boiled down to the cart moving before the extrusion begins because of its viscosity. This makes the solution rather trivial as all that needs to be implemented is a slight extrusion at the start of each print which was added to the prepare function.

Other problems came in the form of hardware error, like errors with the colour sensor detecting the incorrect colour or the motor encoders returning an incorrect value, but all these problems had straightforward solutions. The solution to the colour sensor issue was to simply change the colour detected to a more reliable colour that was less impacted by its surroundings. The solution to the motor encoder issue was to simply create a function that resets the nozzle to the printing surface origin as to mitigate the buildup of error over consecutive uses.

6. Project Management

6.1. Work Break Down

A clear break down of roles and a work timeline is essential for the success of any project or team. It is crucial because of its role as a guide that can be referenced at any time during the project, which is important for managing not only individual team members, but the flow of the project. In the ideal world, a project timeline would be executed perfectly by the ideal team, but problems and other unexpected things can occur that can delay and, in some cases, kill the entirety of a project.

The work breakdown of this project is as follows:

<u>Item Name:</u>	<u>Assignee:</u>	<u>Time (hr)</u>
<u>Pancake Printing Robot (ME 101)</u>	<u>ALL</u>	<u>86</u>
1. <u>Frame Design and Integration</u>	<u>ALL</u>	<u>25</u>
1.1 CAD Modeling of Frame Structure	Dylan L. Calim KW.	5
1.2 Fabrication and Assembly of Frame	Nathan M.	3
1.3 Physical Assembly of 2D Rails	Dylan L.	3
1.4 Physical Assembly of Nozzle Extrusion System	Marwan Z.	2
1.5 Integration of EV3 Brick and Motors	Calim KW. Dylan L.	2
1.6 Final Tests and System Refinement	All	10
2. <u>Nozzle Extrusion System</u>	<u>ALL</u>	<u>12</u>
2.1 Ordering, Measuring, and Assembly Syringe and Tubing	Marwan Z.	1
2.2 CAD Modeling of Tube Mount, Rack-and-Pinion Mechanism, and Extrusion Enclosure	Dylan L. Calim KW.	3

2.3 Fabrication of Rack-and-Pinion Assembly and Nozzle Holder	Nathan M. Dylan L.	1
2.4 Assembly of Extrusion System Components	Nathan M.	1
2.5 Programming Extrusion Rate Control Using EV3 Motor	Calim KW. Marwan Z.	3
2.6 Testing Timing of Program / Rate of Extrusion during Operation	Calim KW. Nathan M.	3
3. <u>2D Rail System for Nozzle Movement</u>	<u>ALL</u>	<u>30</u>
3.1 Research and Selection of Materials and Fabrication Methods	Marwan Z.	3
3.2 CAD Design of Tracks and Motor Attachment Mechanism	Nathan M. Dylan L.	6
3.3 Fabrication of Rail System Components	Nathan M. Dylan L.	2
3.4 Installation of Nozzle on Rail System	Marwan Z. Calim KW.	2
3.5 Implementation of Zeroing Calibration System	Calim KW. Dylan L.	10
3.6 Programming Present Movement Functions for Nozzle Control	Calim KW. Nathan M.	2
3.7 Optimizing Timing of Program for Nozzle Movement in Relation to Rate of Extrusion	Nathan M. Calim KW.	5
4. <u>User Input and Miscellaneous Systems</u>	<u>ALL</u>	<u>9</u>
4.1 Implementation of Colour Sensor	Marwan Z. Nathan M.	2
4.2 Fabrication of Colour Sensor Mounts	Nathan M. Dylan L.	2
4.3 Development of User Interface and Program Selection Logic	Calim KW.	3
4.4 Wiring of Components for Optimal Operation	Marwan Z. Dylan L.	1

4.5 Testing and Altering Programing Screen Outputs	Calim KW. Marwan Z.	1
<u>Contingency Time</u>	<u>N/A</u>	<u>10</u>

Table 4: Work Break Down Structure of Project Along with Respective Assignee

Most tasks completed were done in groups of two team members. This is done to ensure the timeline of the project if one team member is unavailable or preoccupied with other tasks. This, at least specifically for this project, was the safest way to break down the work structure while remaining efficient, given the magnitude of this project accompanied by several external tasks unrelated to the project. The work structure has also been designed to assign tasks to individuals that were more equipped to handle those tasks. Even with the assigning of tasks to specific people, many tasks ended up being a collaborative effort involving the whole team working together. Overall, the structure above was successful in organizing tasks and responsibilities and giving general time estimates for tasks to be completed.

6.2. Project Risk Assessment

As with every project, there will come unforeseen hurdles that vary in impact on the project and its timeline. Table 5 outlines only some of the possible risks taken into consideration prior to beginning major tasks for this project. These risks are broken down into two categories, those being technical and non-technical, and are then evaluated for their respective probability and severity. Risks pretraining to the components of the robot, are classified as technical risks and are usually ranging from medium to high severity. Their severity is due to the massive threat they impose on the project if even slightly malfunctioning. Other risks such as shipping delays and manufacturing machine service are non-technical risks that vary in severity. The impact of shipping delays is completely dependent on the material or component being shipped and its

uniqueness, but by having alternative methods to getting similar components, while prioritizing the shipping of more tailored objects, this risk can become negligible. Severity of machine service is completely dependent on the number of machine outlets available, and naturally the traffic of those outlets, but by having several outlets or several manufacturing methods, this risk can be mitigated.

Risk	Probability (L, M, H)	Severity (L, M, H)	Impact	Mitigation	Back-up Plan
<u>Technical Risks</u>					
Material Failures	L	M	-Rails bending, frame not structurally sound -Collateral damage to other components	-Reinforce and validate using small scale testing of individual components	-Change material selection -Remodel design for better structural integrity
Poor Design	H	M	-Failed prototypes -Increase in costs and duration -Harder to accomplish deadline	-Review prototype design before manufacturing -Design considering context of machinery -Refer to engineering specifications table	-Reiterate and remake design
Material Fatigue from Over Testing	M	H	-Quality of robot functions deteriorating	-Have back up material to	-Swap fatigued component

			-Slow approach to material failure -Material failure close to live demonstration	- swap for testing -Allow for longer rest times in between tests -Have duplicates for components	with back up component
<u>Non-Technical Risks</u>					
Lack Of Resources	L	H	-Failure to complete design	-Keep track of available resources -budget accordingly to account for material waste -include excess material	-Modify design to be completed with remaining resources
Shipping Delays	M	M	-Materials unavailable for processing and fabrication of parts	-Order from trusted sites with quick shipping -Have backup brick-and-mortar store ready	-Go to brick-and-mortar store to purchase materials.
No Manufacturing Machine Service	L	M	-Delays to fabrications and processing of parts -Extended timeline	-Have access to several locations with manufacturing machinery -Utilize other machinery to make similar	-Hand make components using more malleable materials and hand tools

				components until machine repairs	
--	--	--	--	--	--

Table 5: Project Assessment of Technical and Non-Technical Risks

6.3. Progress Schedule

The below figures show the state of the time, during three important stages of the project. Figure 17 shows the state of the timeline initially, while Figure 18 shows the updated timeline a week before the live demonstration with additional tasks, and Figure 19 shows the resultant timeline day of the live demonstration.

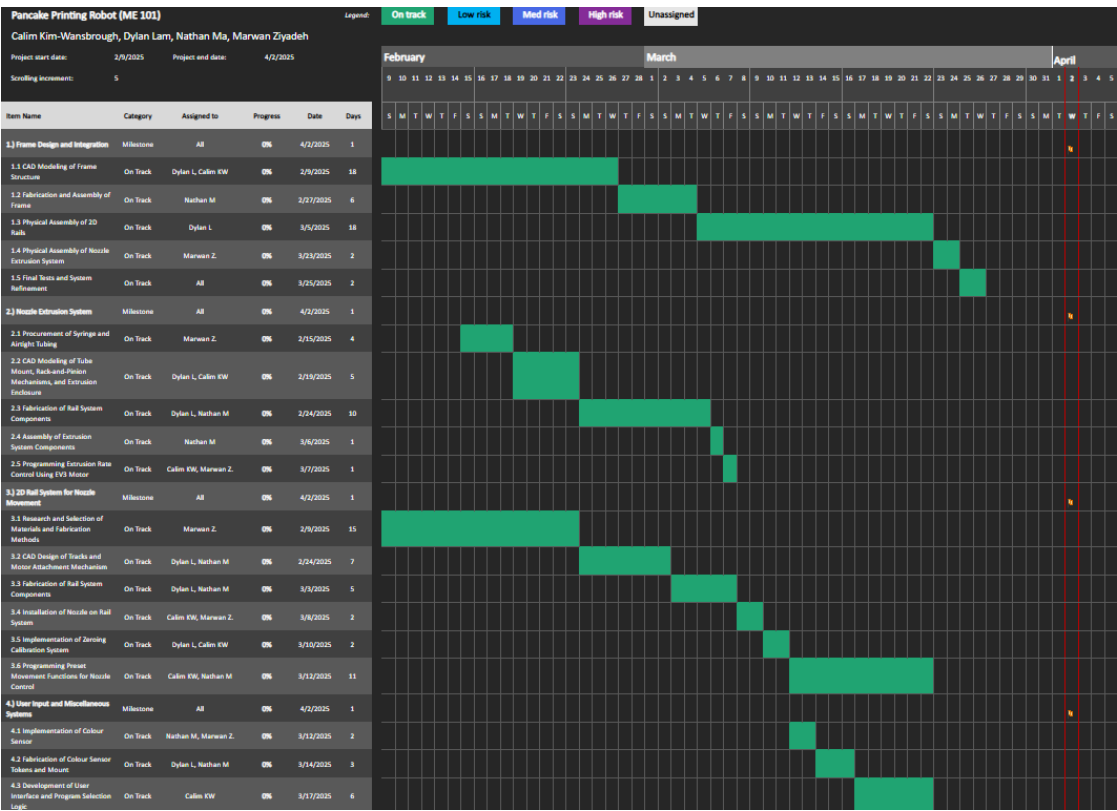


Figure 25: Project Schedule of Original Timeline

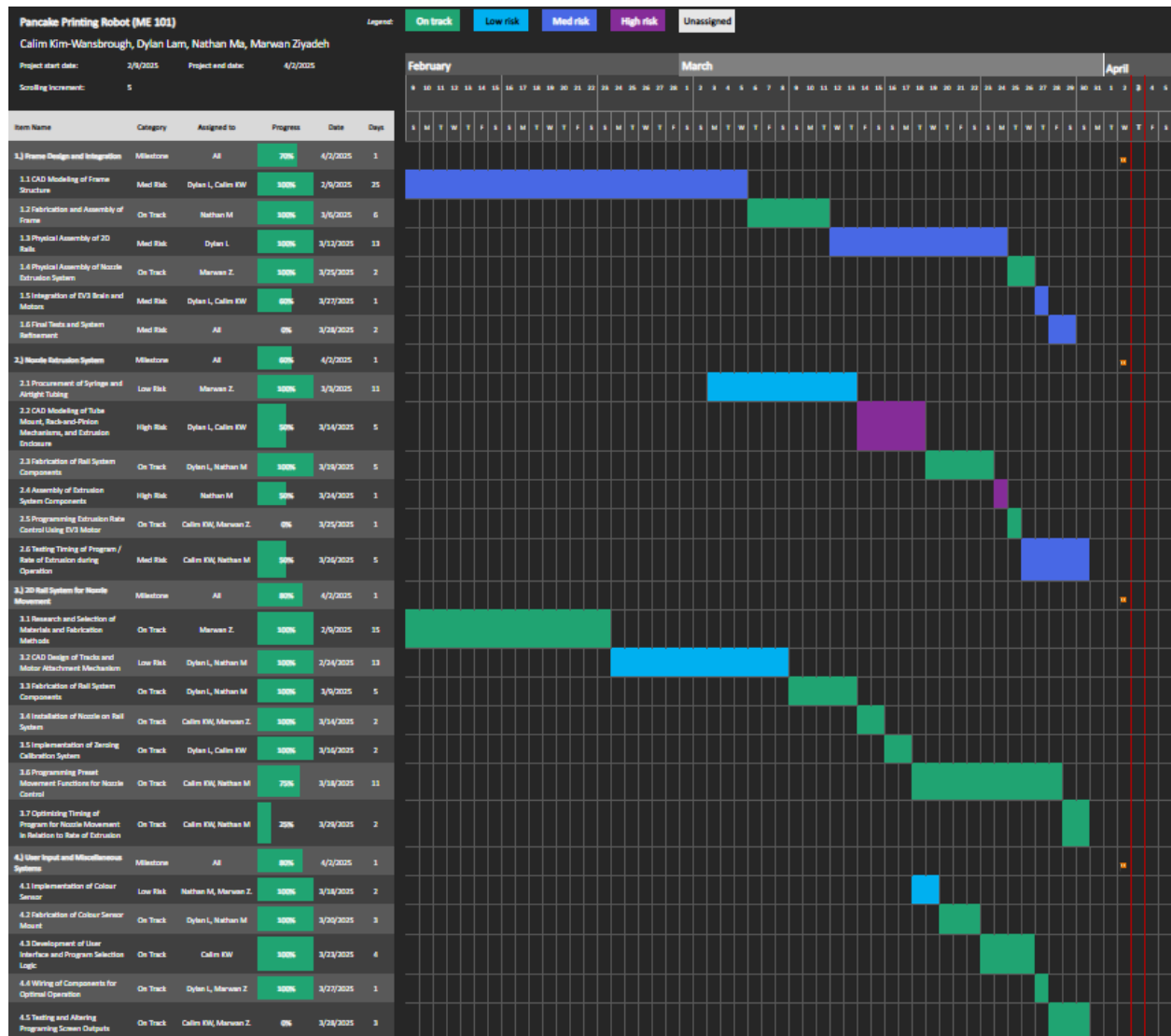


Figure 26: Project Timeline a Week Prior to Live Demonstration

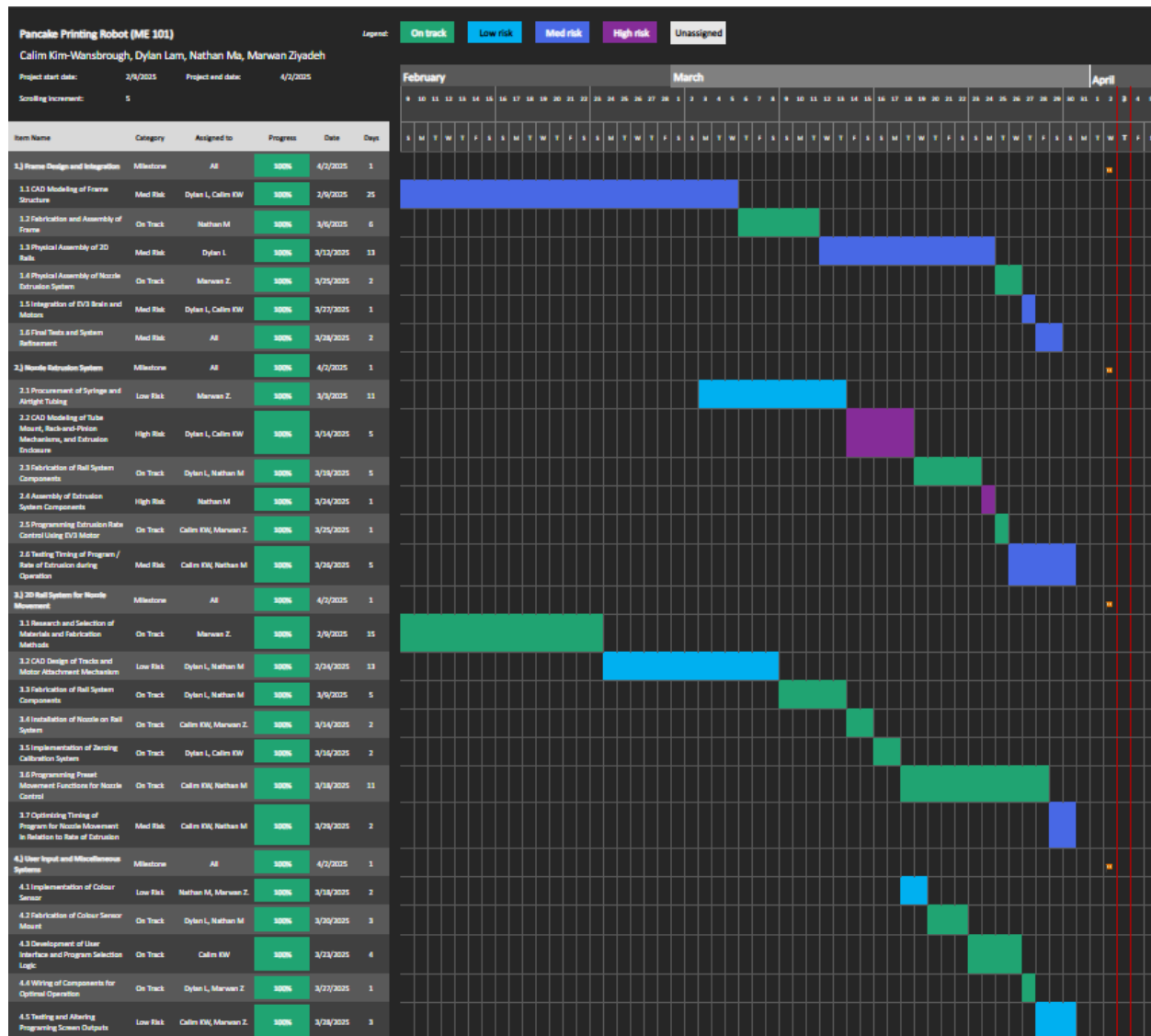


Figure 27: Project Timeline Day of Live Demonstration

6.4. Actual Timeline

The initial timeline shown in Figure 25 gives a list of expected tasks for this project while having an assigned member or members to each one, as well as a start date. The dates next to the milestones are end dates which at the very latest was expected to be completed just prior to the live demonstration deadline of April 2nd 2025. Progressing through the project uncovered a

handful of other tasks that needed to be completed, which were included in the updated timeline shown in Figure 26. That timeline shows the progression of the previously expected tasks including the updated delays accompanied with their respective risk assessment.

Comparing timelines shown in Figure 25 and Figure 26, it is apparent that the duration allotted to certain tasks increased by a week's worth of days. This is solely due to the low project productivity during reading week as other tasks such as preparing for midterms were prioritized over the project. To account for the loss of a weeks' time, certain deadlines for tasks had been pushed forward, allowing for the original deadlines to remain for tasks of greater magnitude.

Also apparent in Figure 26 is the change in risk for certain tasks, denoted by the colour of the bar, meaning the tasks still being incomplete at that current stage possess some potential degree of impact on the overall project. The highest risk that occurred within the project timeline was the development of the extrusion method of the batter. The plan had originally been to extrude the batter using a rack and pinion system that would be applied to the top of the syringe plunger, but the issue with this mechanism was that it was unable to apply enough force on the plunger to properly extrude the batter. For context, the syringe was significantly larger than the typical medical syringe, which already increased the force required for extrusion, but adding on to that the nature of the batter, specifically its viscosity also increasing the force required for any extrusion let alone consistent extrusion. Due to the unpredicted amount of force for extrusion, the original idea needed to be scrapped resulting in the rack-and-pinion mechanism being replaced with the lead screw. Upon testing, the lead screw mechanism had outperformed the rack and pinion in terms of quality of extrusion, resulting in cleaner and consistent prints during trial runs.

Another dire risk experienced during the period of this project was the non-functioning laser cutting machine at the Rapid Prototyping Centre (RPC) in Waterloo. Due to this unforeseen

circumstance the development of the rails was delayed resulting in medium level impact on the timeline of this project. The reason this circumstance did not result in a greater impact is purely due to the simplistic design of the rail allowing for the temporary substitution for rails made with other materials. The lack of service for the laser cutter did contribute to the overall delay in timeline, but not to the degree of either reading week or the redesign of the nozzle mechanism.

Figure 27 shows the final state of the timeline just prior to the live demonstration, by now, the robot had been fully assembled and successfully performed most of its expected tasks to an acceptable quality. Few more tasks since the timeline shown in Figure 26 had been delayed or even altered, resulting in some notable impact on the project. Those tasks being one of the colour-based prints, specifically the face print which had been changed to a spiral shape as mentioned earlier, and calibrating of the extrusion rate of the nozzle. The hurdles of that arose from these tasks, all stemmed from the redesign of the nozzle extruding mechanism, as they are all dependent the nozzles' ability to extrude consistently.

7. Conclusion

7.1 Mechanical Portion

The key features of this robot were the extruding of the pancake batter, and the mobility provided by gantry and cart movement systems. The two rail systems accompanied by the lead screw mechanism were responsible for these primary functions. Through the design process, there were numerous instances where an idea had to be improved or redesigned completely. For the extrusion system, a rack and pinion were initially used to extrude the pancake batter. The issue was that the rack and pinion would repetitively skip teeth and would fail to reliably extrude

batter. It was discovered that a rack and pinion doesn't operate well in high torque scenarios. To address this, the extrusion mechanism was switched to use a lead screw which much more reliably extruded batter. Despite the extrusion mechanism operating as intended, the batter extrusion still had issues due to its inconsistent flow rate due to the viscous quality of the batter.

The gantry system was built on a sturdy Tetrix frame, on which all the Lego EV3 parts were mounted. 3D printing was used to interface all the Lego EV3 parts to the Tetrix frame securely. Laser cutting was used to create long, flat, and uniform rails for the gantry. All seven engineering specifications were successfully verified during live demonstration day. These ensured that the mechanical design was able to print pancake batter while being easy to use, reliable, lightweight, and small.

7.2 Programming Portion

The primary programming objectives for this robot were all timing and accuracy based. The objectives for the programming portion of the robot were whether it was able move the nozzle cart in the correct sequence printing the desired shape to an accurate depiction, and whether the robot was able to extrude sufficient batter such that all parts of the shapes were drawn with a consistent amount of batter. Other high priority objectives were to correctly associate a detected colour, through the colour sensor, with the correct shape, as well as to move the nozzle in the correct direction while manual mode was activated and in use. Some lower priority programming objectives include providing clear instructions on operation for the user.

Evaluating the programming of the robot, majority of the objectives were able to be completed to a more than acceptable level. After several trial runs, the timing of the extrusion in relation to the movement of the nozzle cart was perfected allowing for consistent extrusion, only

when the batter had been diluted to a runnier consistency. The colour sensor was functioning correctly as well, confirming then printing the correctly associated shape when certain colours were shown to the sensor. As for manual mode, the program was again successful in moving the nozzle cart in the direction specified by the user, with relatively minor delay. The communication between the robot and the user had also been successful, through user testing, the instructions were mostly clear allowing for most users to use the robot without significant difficulty.

8. Recommendations

8.1. Mechanical Design Recommendations

Despite the successful outcome of this project, there is still plenty of room for improvements to be made. A weaker area of this project is the extrusion mechanism, the overall maintainability of the robot, and the wheel and rail gantry system.

One reoccurring issue during testing was the batter's consistency; it was often too runny or too thick. When it was too runny, the extrusion mechanism could extrude but could not stop extruding due to the batter simply dripping from the nozzle. When it was too thick, the extrusion mechanism could not extrude anything due to the high cohesive and adhesive forces. Several potential solutions could address this issue while maintaining most of the existing mechanical design. Replacing the syringe with one that has a smaller plunger would reduce the force required to extrude batter. Similarly, replacing the tube with a larger one would also reduce the force required to extrude the batter. Alternatively, replacing the medium torque motor with a high torque motor would also help produce more force to extrude the batter. By using a thicker

batter, reducing force required to extrude, and increasing force produced, the extrusion mechanism could achieve better performance while minimize issues with unintended dripping

Another prominent weakness with the design was the maintainability of the robot. This stems from several factors, including poor cable management, a relatively small and compact frame, and the oversized syringe used in the extrusion mechanism. To address these issues, both the vertical and horizontal beams could be increased in size to create more free space for mounting sensors and routing cables. This would improve accessibility and simplify maintenance tasks. Additionally, more time and effort can be spent redesigning the extrusion mechanism to be easily disassembled, cleaned, and refilled, while maintaining its current robustness. These adjustments would improve the robots long term maintainability and save a lot of time during testing.

Lastly, major improvements can be made to the wheel and rail gantry system. The off-road wheels were initially selected due to limited availability of more suitable alternatives. smaller wheels with a more predictable profile would be preferable, as they offer a smaller footprint and are compatible with narrower rails. Additionally, the rails themselves could be redesigned to be lower profile. The current design features bulky rails, which contribute to the robot's overall unwieldiness and fragility, making it difficult to move and work on.

8.2. Software Design Recommendations

On the software side of the robot all the programming did function as expected however changes to code could optimize the program and printing process. The first recommendation is the way the code handles diagonals. Diagonals are tricky to implement as they require the two sets of motors to operate at different speeds to ensure that the nozzle reaches the x and y position

at the same time. To achieve this either parametric equations or simple trigonometric equations will have to be used. The use of either of these can lead to other issues involving the EV3 motors only taking power input rather than a speed input however it would open the door to more complex designs.

Another improvement that can be made is the ability for the user to import custom designs through text files. The current version of our robot only allows for customizability through manual mode. However, being able to upload a text file that contains ones and zeroes to indicate where batter should go would greatly improve the ability to create new designs with a higher level of precision through autonomy. A limitation of this could be the EV3's limited processing power, however with good optimization implementing this feature is quite feasible.

A significant weakness of this printer is its slow speed. The speed was capped very low for two reasons. The first is the inaccuracy of the motor encoders and the second was the slow extrusion rate. the slow extrusion rate could easily be solved by using a different lead screw which has threads further apart to allow for more distance per rotation. The inaccuracy of the motor encoders on the other hand needs a more elegant control system to be implemented in order to allow for higher print speeds. The recommended control system is a PID. PID's can correct overshoot and are able to work rather quickly as a closed loop system. Additionally, if a PID system is to be implemented a link between the extrusion rate and the current speed of the cart should be added to ensure that as it slows down certain areas don't have too much batter printed on them.

While this version of the pancake printer can be considered a success there are still a vast number of improvements that can be made on the software side that would improve the functionality and user experience.

Appendix A – Pancake Printer Code

```
File: C:\Users\marwa\Downloads\Pancake Printer ME101.c

/*
Group 6
*/

/*
Documentation:
Y Position based on Motor port A and B
X Position based on Motor port C
Extrusion based on Motor port D

Sensor port 1 is used for touch
Sensor port 3 is used for color
Sensor port 2 and 4 were bound to the ultrasonic and gyro respectively however were unused.

Radius of the wheels used are 2.75cm hence the formula found in DEG2DIST WHEEL
EXTRUDE_POWER is based on the extrusion substance. Water is 10 and batter is 100
BED_MAX & BED_MIN are to ensure extruding occurs during certain times
TOLERANCE was put in place to prevent ocillation due to poor motor control
*/

const float DEG2DIST_WHEEL = PI*2.75/180;
const float EXTRUDE_POWER = 10;
const float SPEED_X = 10;
const float SPEED_Y = 10;
const float TOLERANCE = 0.1;
const float BED_MIN = -1;
const float BED_MAX = 25;

//Configuring sensors, function provided in class
void configureAllSensors()
{
    SensorType[S1] = sensorEV3_Touch;
    SensorType[S2] = sensorEV3_Ultrasonic;
    SensorType[S3] = sensorEV3_Color;
    wait1Msec(50);
    SensorMode[S3] = modeEV3Color_Color;
    wait1Msec(50);
    SensorType[S4] = sensorEV3_Gyro;
    wait1Msec(50);
    SensorMode[S4] = modeEV3Gyro_Calibration;
    wait1Msec(100);
    SensorMode[S4] = modeEV3Gyro_RateAndAngle;
    wait1Msec(50);
}

//function returns the current y position relative to origin in centimeters
float getY()
{
    return DEG2DIST_WHEEL*nMotorEncoder[motorA];
}

//function returns the current x position relative to origin in centimeters
float getX()
{
    return DEG2DIST_WHEEL*nMotorEncoder[motorC];
}

//will toggle the extrusion of the batter based on a parameter.
void extrude(bool on)
{
    if(on)
    {
        motor[motorD] = EXTRUDE_POWER;
    }
    else
    {
        motor[motorD] = 0;
    }
}
```

File: C:\Users\marwa\Downloads\Pancake Printer ME101.c

```
}

void move_Both(float x, float y, float ex_minX, float ex_maxX, float ex_minY, float ex_maxY)
{
    while(abs(x-getX()) > TOLERANCE || abs(y-getY()) > TOLERANCE)
    {
        if(ex_minX == ex_maxX)
        {
            ex_minX = BED_MIN; ex_maxX = BED_MAX;
        }
        if(ex_minY == ex_maxY)
        {
            ex_minY = BED_MIN; ex_maxY = BED_MAX;
        }
        int directionX = 0, directionY = 0;
        if(x-getX() > 0)
        {
            directionX = 1;
        }
        else
        {
            directionX = -1;
        }
        if(y-getY() > 0)
        {
            directionY = 1;
        }
        else
        {
            directionY = -1;
        }
        //if it is "close enough" to the desired x or y position it stops moving in that direction
        if(abs(x-getX()) < TOLERANCE)
        {
            directionX = 0;
        }
        if(abs(y-getY()) < TOLERANCE)
        {
            directionY = 0;
        }
        motor[motorC] = directionX * SPEED_X;
        motor[motorA] = directionY * SPEED_Y;
        motor[motorB] = directionY * SPEED_Y;
        //if it is within the extrusion bounds it toggles the extrusion.
        if(getX() >= ex_minX && getX() <= ex_maxX && getY() >= ex_minY && getY() <= ex_maxY)
        {
            extrude(true);
        }
        else extrude(false);
    }
    //safeguards, ensures that movement and extrusion is completely stopped before moving on from
    motor[motorA] = 0;
    motor[motorB] = 0;
    motor[motorC] = 0;
    extrude(false);
    waitMsec(100);
}

//moves the nozzle to a given position
void prepare(int x, int y)
{
    move_Both(x,y, 1, 0, 1, 0);
    //extrudes a little bit to ensure that the batter is at the tip of the nozzle ready to print
    extrude(true);
    waitMsec(500);
    extrude(false);
}
```

File: C:\Users\marwa\Downloads\Pancake Printer ME101.c

```
//shoves the robot into the corner (origin) and sets that as the new 0s for the motor encoders
void reset()
{
    prepare(0,0);
    motor[motorA] = motor[motorB] = motor[motorC] = -10;
    wait1Msec(1000);
    motor[motorA] = motor[motorB] = motor[motorC] = 0;
    nMotorEncoder[motorA] = 0;
    nMotorEncoder[motorC] = 0;
}

//square function, uses four corner points and draws lines between them
void square()
{
    prepare(4,2);
    move_Both(4, 16, 4, 4, 2, 16);
    move_Both(18, 16, 4, 18, 16, 16);
    move_Both(18, 2, 18, 18, 2, 16);
    move_Both(4, 2, 4, 18, 2, 2);
    reset();
}

//triangle function maps out 3 points and drives between them
void triangle()
{
    prepare(4,2);
    move_Both(18, 16, 4, 18, 2, 16);
    move_Both(18, 2, 18, 18, 2, 16);
    move_Both(4, 2, 4, 18, 2, 2);
    reset();
}

void spiral()
{
    prepare(4, 2);
    move_Both(19, 2, 4, 19, 2, 2);
    move_Both(19, 15, 19, 19, 2, 15);
    move_Both(6, 15, 6, 19, 15, 15);
    move_Both(6, 6, 6, 6, 6, 15);
    move_Both(15, 6, 6, 15, 6, 6);
    move_Both(15, 12, 15, 15, 6, 12);
    move_Both(10, 12, 10, 15, 12, 12);
    move_Both(10, 9, 10, 10, 9, 12);
    reset();
}

void manual(bool & on)
{
    //things for the brain to display (instructions, current position)
    displayString(3, "Manual mode");
    displayString(4, "Use pad to operate");
    displayString(5, "Hold touch sensor to return");
    displayString(6, "X: %2.2f, Y: %2.2f", getX(), getY());
    bool manualMode = true;
    while(manualMode)
    {
        //if touch sensor is hit then it exits this while loop
        if(SensorValue[S1])
        {
            extrude(false);
            eraseDisplay();
            reset();
            manualMode = false;
        }
        //if enter is pressed it switches the states of the extrusion
        if(getButtonPress(buttonEnter))
        {
            while(getButtonPress(buttonEnter)){}
        }
    }
}
```

Page 3 of 5

File: C:\Users\marwa\Downloads\Pancake Printer ME101.c

```
    on = !on;
    extrude(on);
    wait1Msec(200);
}
//if a button is being pressed an it isnt pressing up against the bounds it will turn the
if(getButtonPress(buttonUp) && getY() < 19)
{
    motor[motorA] = SPEED_Y;
    motor[motorB] = SPEED_Y;
}

else if(getButtonPress(buttonDown) && getY() > 0)
{
    motor[motorA] = -SPEED_Y;
    motor[motorB] = -SPEED_Y;
}
else
{
    motor[motorA] = 0;
    motor[motorB] = 0;
}
if(getButtonPress(buttonRight) && getX() < 24)
{
    motor[motorC] = SPEED_X;
}
else if(getButtonPress(buttonLeft) && getX() > 1)
{
    motor[motorC] = -SPEED_X;
}
else
{
    motor[motorC] = 0;
}
}
eraseDisplay();
}

//a way to confirm if the color scan was intended
int confirm(string message)
{
    displayString(3, "Printing %s", message);
    displayString(4, "Press enter to confirm");
    int num = 0;
    while(!getButtonPress(buttonAny))
    {
    }
    if(getButtonPress(buttonEnter))
    {
        num = 1;
        displayString(3, "Now Printing %s", message);
        displayString(4, "Please Keep hands away");
    }
    else
    {
        eraseDisplay();
    }
    return num;
}

//function which maps different scanned colors to different functions
void colorShape(bool & system)
{
    string message = "";
    if(SensorValue[S3] == (int)colorRed)
    {
        message = "square";
        if(confirm(message))
        {
            square();
            eraseDisplay();
        }
    }
}
```

Page 4 of 5

File: C:\Users\marwa\Downloads\Pancake Printer ME101.c

```
    }
}
else if(SensorValue[S3] == (int)colorBlue)
{
    message = "triangle";
    if(confirm(message))
    {
        triangle();
        eraseDisplay();
    }
}
else if(SensorValue[S3] == (int)colorBlack)
{
    message = "spiral";
    if(confirm(message))
    {
        spiral();
        eraseDisplay();
    }
}
else if(SensorValue[S3] == (int)colorWhite)
{
    displayString(3, "Are you sure you want to exit");
    displayString(4, "Press enter to confirm");
    while(!getButtonPress(buttonAny){})
    if(getButtonPress(buttonEnter))
    {
        displayString(3, "Exiting Program");
        system = false;
    }
    eraseDisplay();
}
}

task main()
{
    configureAllSensors();
    //24x19 bed size
    bool system = true;
    //the main loop which enables the program to run/be broken out
    while(system)
    {
        displayString(3, "WAITING FOR INPUT");
        displayString(4, "KEYPAD FOR MANUAL");
        displayString(5, "TOUCH SENSOR FOR AUTOMATIC");
        bool manualExtrude = false;
        while(!getButtonPress(buttonAny) && !SensorValue[S1])
        {}
        eraseDisplay();
        //based on if touch was pressed or the keypad a different mode will be entered
        if(SensorValue[S1])
        {
            colorShape(system);
        }
        else
        {
            manual(manualExtrude);
        }
        wait1Msec(1000);
    }
    displayString(3, "Exiting program");
    displayString(4, "Thanks for using!");
    reset();
    wait1Msec(3000);
}
```